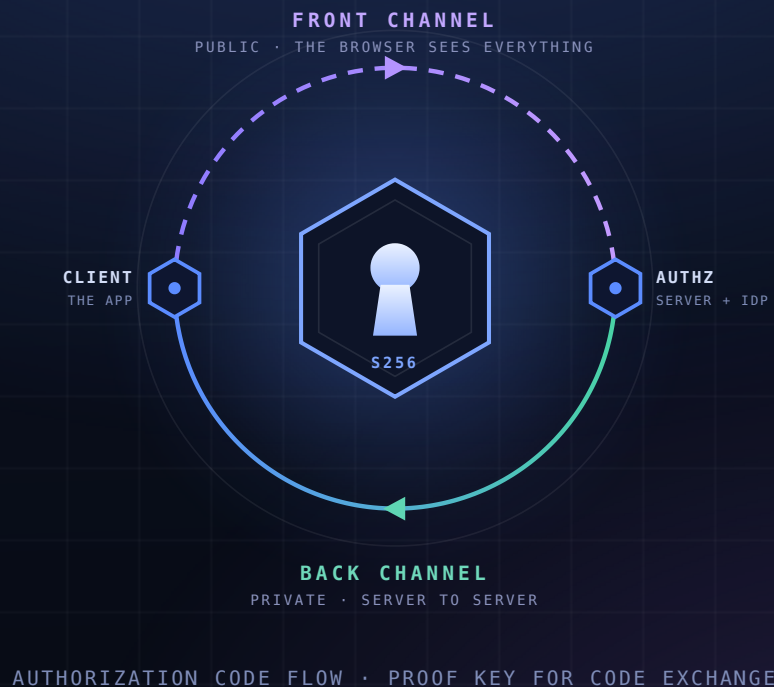


THE ULTIMATE DEVELOPER'S GUIDE TO

OAUTH 2.1 and PKCE

Delegated authorization from first principles — the flow, the cryptography, the attacks each rule prevents, and a runnable simulator to take it all apart.



15 hands-on labs · the complete source of an OAuth 2.1 server, API and client · in the book

AUTHOR

Smiroh Dev

Smiroh Publishers
FIRST EDITION

The Ultimate Developer's Guide to OAUTH 2.1 and PKCE

Smiroh Dev

PUBLISHER	Smiroh Publishers
EDITION	First edition
PUBLISHED	September 2026
FORMAT	Electronic book (PDF), 81 pages
ISBN (EB00K)	To be assigned
SUBJECT	Computers / Security / Web Development

Copyright © 2026 Smiroh Dev. All rights reserved.

Published by Smiroh Publishers. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, or other electronic or mechanical methods — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

TRADEMARKS

All trademarks and registered trademarks appearing in this book are the property of their respective owners. They are used editorially, to the benefit of the trademark owner, with no intention of infringement. The publisher is not associated with any product or vendor mentioned in this book.

STANDARDS REFERENCED

This book discusses OAuth 2.1 and related specifications published by the Internet Engineering Task Force (IETF) and the OpenID Foundation. Those documents are the normative sources; where this book and a specification disagree, the specification governs. A complete reading list appears in the final chapter.

LIMIT OF LIABILITY AND DISCLAIMER OF WARRANTY

The information in this book is provided on an “as is” basis, without warranty of any kind, express or implied. While every precaution has been taken in its preparation, neither the author nor the publisher shall have any liability to any person or entity with respect to any loss or damage caused, or alleged to have been caused, directly or indirectly by the information contained in it. Security is a moving field: verify any practice described here against the current specifications and your own threat model before relying on it.

ABOUT THE COMPANION SIMULATOR

The software accompanying this book is a teaching tool. It deliberately implements insecure configurations so that they can be demonstrated, generates ephemeral keys, keeps all state in memory, and serves plain HTTP on `localhost`. It must never be deployed, exposed to a network, or used to protect anything real. Code listings are illustrative; production systems should use maintained, independently audited OAuth libraries and authorization servers.

What is in this guide

01	The problem OAuth solves	Why handing over your password is the wrong answer	4
02	The four roles	Resource owner, client, authorization server, resource server	5
03	Front channel and back channel	The one distinction it is built on; which grant uses which	6
04	The Authorization Code flow	All eleven steps, drawn and then narrated	9
05	The attack PKCE prevents	What a stolen authorization code used to be worth	13
06	PKCE, precisely	Three lines of arithmetic, and why they are enough	14
07	Anatomy of a token	Inside a JWT; access vs ID vs refresh; opaque vs JWT	16
08	OpenID Connect: who is this?	The identity layer: ID tokens, nonce, /userinfo	20
09	Scopes and consent	How authority is narrowed, three times over	24
10	Refresh tokens and rotation	Long sessions from short-lived tokens	25
11	What OAuth 2.1 changed	Mostly a list of deletions, and that is the point	26
12	How an API validates a token	Four checks, in order, on every request	27
13	The mistakes that matter	A checklist drawn from real incidents	28
14	Run it yourself	Every concept here, as a runnable lab	29
15	OAuth 2.1 for agentic AI	Agents, delegation chains and structured limits	30

Part II — The Labs

—	Setup, and how to run it	36	A	crypto-lab.js	44
1–3	The flow, and the proof	37	B	authz-server.js	47
4–6	Breaking the rules	38	C	api-server.js	62
7–8	Scope, and token confusion	39	D	client.js	67
9–10	Machine tokens; revocation	40		1,933 lines, complete	
11	OpenID Connect	41			
12–13	An agent, and its limits	42			
14–15	Delegation chains	43			
—	Glossary	Every term in one place			80
—	The specifications	Which RFC says what, and which to read first			81

The problem OAuth solves

An application needs data that belongs to you, held by somebody else. There is an obvious way to arrange that, and it is a catastrophe.

Suppose a budgeting app wants to read your bank transactions. The naive design is to ask you for your banking username and password and log in as you. Every single thing wrong with that idea turns out to be something OAuth fixes.

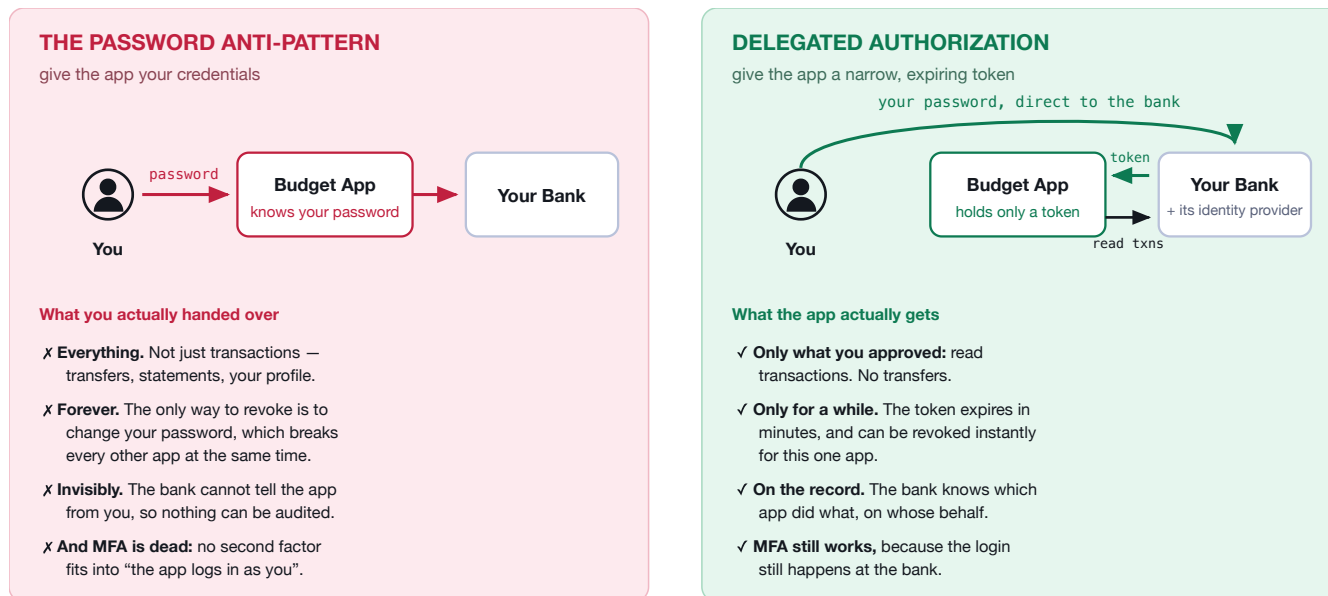


Figure 1.1 — The same goal, two arrangements. On the right, the password travels only between you and the party that issued it; the application is handed a credential that is narrow by construction.

The one-sentence definition

OAuth 2.1 is a protocol for letting an application act on your behalf, in a limited way, without ever seeing your password. You authenticate to the party that already knows you; that party issues the application a scoped, expiring, revocable token.

The rest of this guide is about doing that *safely* when the exchange must pass through a web browser — the single constraint, explored in Chapter 3, that makes OAuth look harder than the sentence above.

Three things OAuth is not

Not authentication

An access token says *what may be done*, not *who is here*. Treating one as proof of identity is a vulnerability class of its own; identity is a separate token (Chapter 7).

Not encryption

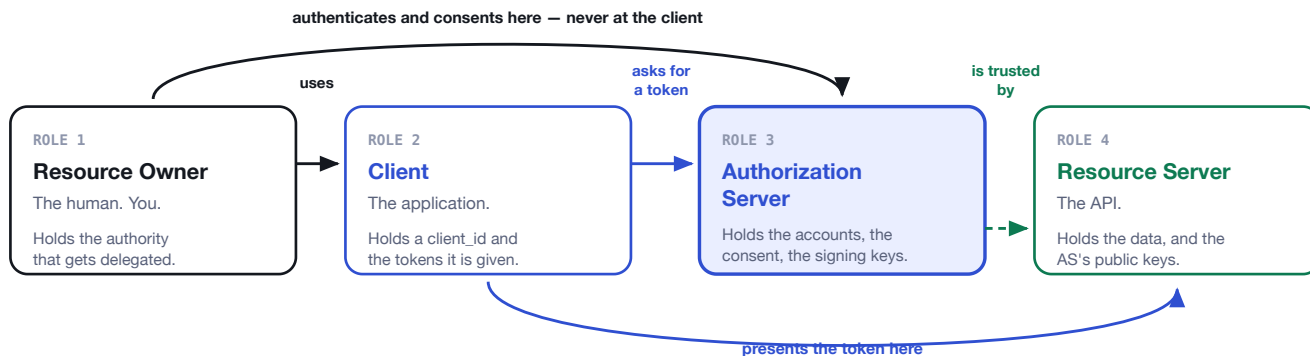
A bearer token is a password-equivalent in plain text: anyone who copies it can use it. OAuth assumes **TLS everywhere**.

Not a permission system

Scopes are coarse labels on an app's authority. Whether *this user* may touch *this record* stays your API's decision, on every request.

The four roles

OAuth defines four participants. Almost every confusing sentence in the specifications becomes clear once you know which of the four is being talked about.



The asymmetry that makes it work:

the **client** never learns who you are, and the **API** never learns how you proved it. Each knows only what it needs.

Figure 2.1 — The four roles and the relationships between them. Roles 3 and 4 are often the same company but never the same code path: issuing tokens and accepting them are different jobs.

The same four roles, three ways

ROLE	BANK EXAMPLE	“SIGN IN WITH GOOGLE”	IN THE SIMULATOR
Resource Owner	You, the account holder	You, the Google user	alice or bob
Client	The budgeting app	The site you are signing in to	:9020
Authorization Server	The bank's login service	Google's OAuth service	:9000
Resource Server	The bank's accounts API	Gmail / Drive / Calendar APIs	:9010

Two kinds of client, and why it matters so much

This distinction decides how a client may authenticate, and it is why PKCE exists.

Confidential client HAS A SECRET

Code that runs on a server you control. It can keep a `client_secret` because nobody can read its memory or disk.

Examples: a server-rendered web app, a backend service.
Proves identity by: sending its secret to the token endpoint.

Public client CANNOT KEEP A SECRET

Code that runs on someone else's device. Anyone can read the JavaScript bundle or decompile the binary, so any embedded secret is published to the world.

Examples: a single-page app, a mobile app, a CLI. **Proves identity by:** PKCE, and nothing else.

The mistake to avoid. A single-page app is a **public** client however private your bundle feels: a secret shipped to a browser or an app store is not a secret. OAuth 2.1's answer is to stop depending on one — PKCE is mandatory for every client, so correctness no longer rests on this classification.

Front channel and back channel

If you remember one idea from this guide, make it this one. Every rule in OAuth 2.1 — including all of PKCE — follows from the difference between these two ways of moving data.

The client and the authorization server need to talk twice, and the two conversations travel by completely different routes with completely different security properties. They are not alternatives — in the Authorization Code flow you use **both, in order**. The next page sets out exactly which grants use which.

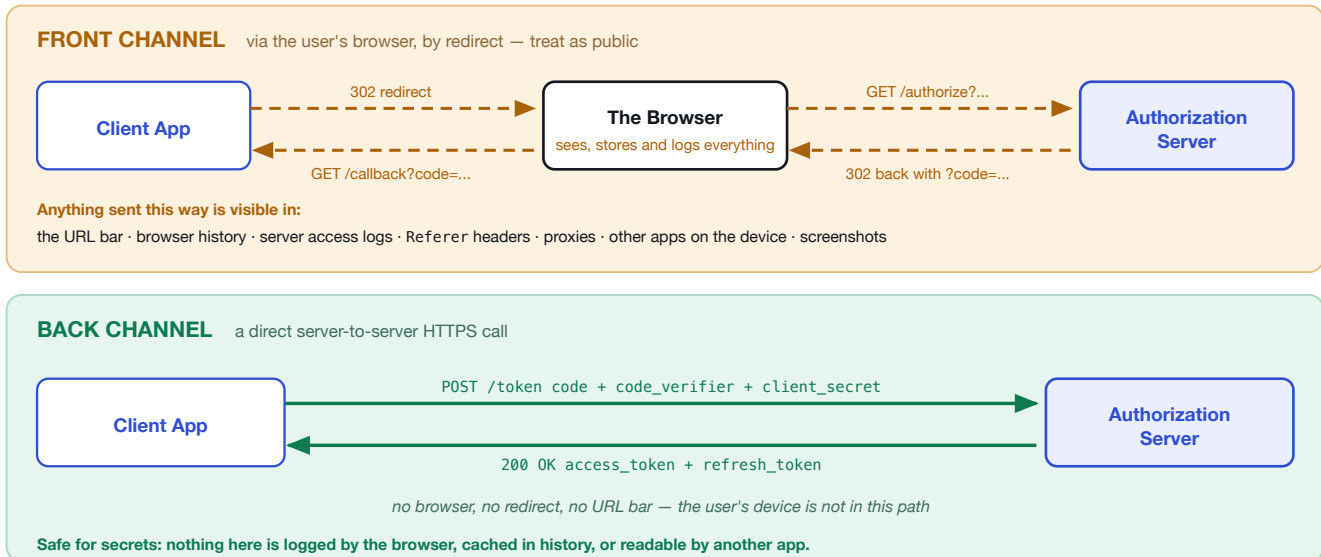


Figure 3.1 — The same two parties, two channels. Dashed arrows are browser redirects; solid arrows are direct HTTPS calls.

The rule that follows

May cross the front channel

- `client_id` — public by definition
- `redirect_uri`, `scope`, `state`
- `code_challenge` — a hash, not a secret
- the `code` — useless on its own (*this is the whole trick*)

Must never cross it

- `client_secret`
- `code_verifier`
- `access_token`, `refresh_token`, `id_token`
- the user's password, obviously

This is why the implicit grant was removed. It returned an *access token* on the front channel, in the URL fragment — landing it in browser history and in any Referer header the page emitted. Chapter 11 has the full list of what OAuth 2.1 deleted and why.

“Front channel” does not mean unencrypted

Both channels run over TLS in production. The front channel is dangerous not because it is exposed in transit, but because of where the data **comes to rest**: browsers write URLs into history, servers into access logs, pages leak them through `Referer`. TLS does nothing about any of that. OAuth 2.1 still requires `https` everywhere, with two carve-outs — `http://localhost` for development, and private-use schemes like `com.example.app:/callback` for native apps. That is why the simulator may run on plain HTTP, and why nothing else should.

Do you need both?

Three questions this trips people on: are the channels alternatives, which grant uses which, and does the answer depend on who owns the application. Taken in order.

The short answer. They are **not** alternatives. In the Authorization Code flow both are used, **always, in this order**: the front channel obtains the code, then the back channel exchanges it for tokens. Neither half is optional and neither can do the other's job. Some *other* grants use the back channel alone — and nothing safe uses the front channel alone, which is precisely what OAuth 2.1 removed.

Which grant uses which channel

GRANT	FRONT	BACK	WHEN YOU USE IT
Authorization Code + PKCE	yes	yes	A user is present and must approve. Both channels, in sequence. This is the default answer for almost everything.
Refresh token	no	yes	Renewing an expired access token later, with no user present. The front channel already did its job at the original authorization.
Client credentials	no	yes	A nightly batch job or service-to-service call. No user exists , so there is nobody to redirect and nothing to consent to.
Device grant	no*	yes	A CLI, a TV, an agent. The client itself only ever uses the back channel; the human approves on a <i>separate</i> device. See Chapter 15.
Introspection, revocation	no	yes	Administrative calls about a token. Always server to server.
Implicit (removed)	yes	no	Returned a token on the front channel with no back-channel step at all. Front channel alone is the thing OAuth 2.1 deleted.

* The device grant never redirects the client, so it has no front channel of its own. The human's approval is a browser interaction, but on a different device and outside the client's flow entirely.

One connection, both channels, in order

Sarah connects a budgeting app to her bank. Watch which channel carries each step, and what is on it:

- FRONT** The app redirects Sarah's **browser** to the bank. On the URL: `client_id`, `scope`, `state`, `code_challenge`. Nothing secret.
- FRONT** Sarah signs in **at the bank** and approves. The bank redirects her browser back to the app carrying a `code` — which is useless on its own.
- BACK** The app's **server** posts that code, plus its `code_verifier` and secret, straight to the bank's `/token` endpoint. Sarah's browser never sees this call, and never sees the tokens it returns.
- BACK** Five minutes later the access token expires. The app's server refreshes it — **back channel only**. Sarah is asleep; no browser is involved, and none is needed.

Steps 1–2 cannot be done on the back channel: only a browser can show Sarah a login screen. Steps 3–4 must not be done on the front channel: they carry secrets. That is the whole reason there are two.

First party, third party, and why it changes nothing

The last of the three questions, and the one that causes the most trouble in design reviews: does any of this depend on who owns the application?

Who owns the client is a different question

The back channel is defined by the **network path** — a direct server-to-server call that does not pass through anybody's browser. It says nothing about who wrote the application. Both of these are back-channel calls:

Third party BACK CHANNEL

A budgeting app built by a different company POSTs to your bank's `/token` endpoint from its own server. Nothing is shared but a registered `client_id` and a secret.

First party BACK CHANNEL

Your company's own web app POSTs to your company's own authorization server. Same wire, same rules, same PKCE requirement.

Three properties get conflated constantly. They are independent, and only the first one is about channels:

PROPERTY	THE QUESTION IT ANSWERS	WHAT IT ACTUALLY CHANGES
Channel	Does it pass through the browser?	What may travel on it. Nothing secret on the front channel, ever.
Client type	Can this code keep a secret?	Whether it authenticates at the token endpoint. A question of <i>deployment</i> , not ownership — a third-party server app is confidential; your own SPA is public.
Ownership	Who wrote it?	Registration and trust policy — chiefly whether you skip the consent screen for your own apps. It changes no protocol rule.

The trap this clears up. “Back channel” does not mean “internal” or “trusted”, and “front channel” does not mean “third party”. A third-party integration uses both channels; so does your own first-party web app. The only question a channel answers is whether the user's browser can see the data.

The Authorization Code flow

All of OAuth 2.1 in one picture; the next two pages walk it step by step.

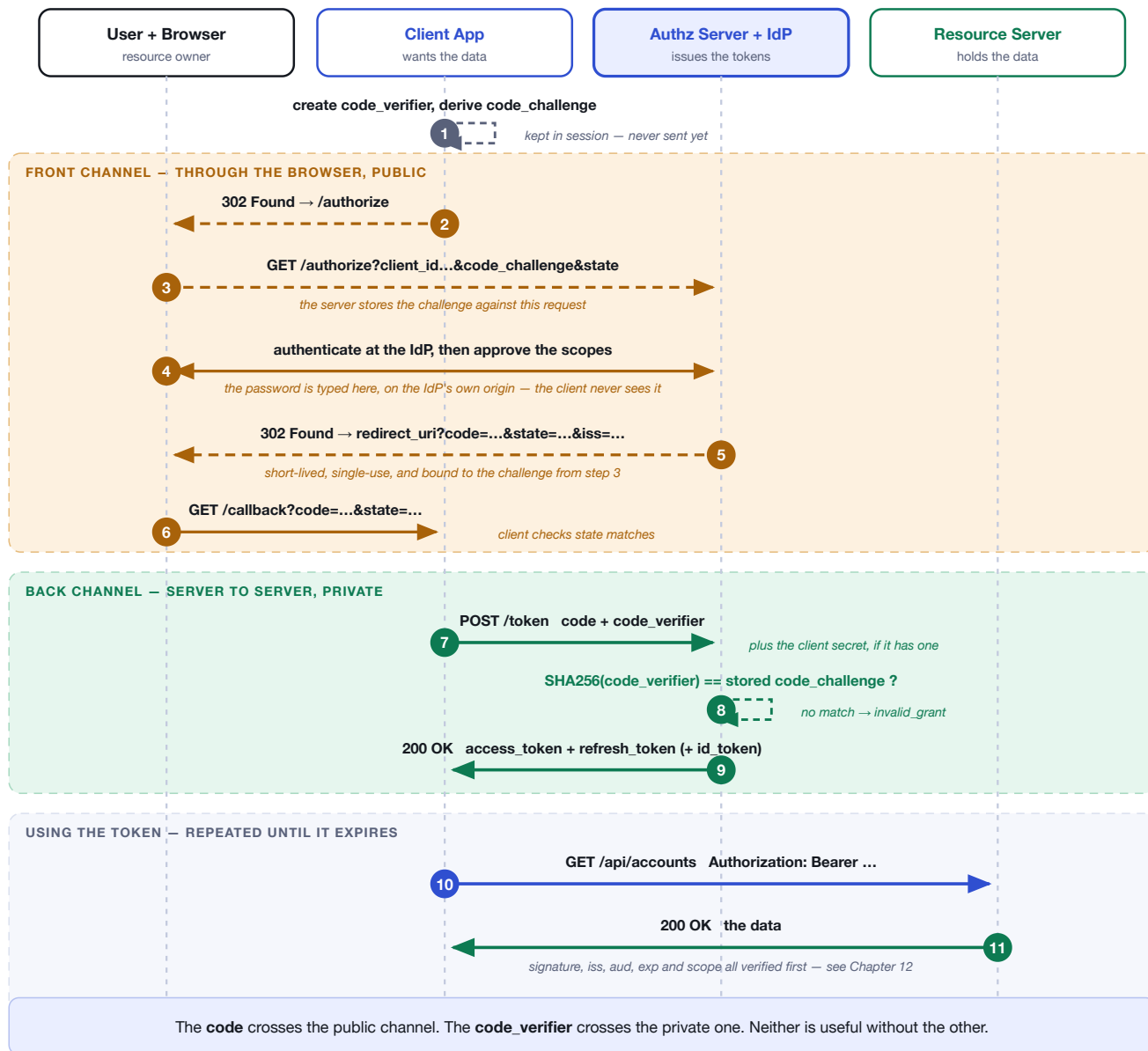


Figure 4.1 — Dashed arrows are browser redirects, solid arrows are direct HTTP calls, looped arrows are local computation. Steps 10–11 repeat until the token expires.

Three things to notice

The user is a participant

Steps 2–6 happen in a browser because a human has to be asked. That is not overhead — it is the consent.

The code is a claim ticket

It is not a token and grants nothing: a single-use reference, valid about a minute, redeemable only by whoever started the flow.

Two channels, one flow

The split at step 7 is the design: the risky public hop carries something worthless, the valuable exchange happens out of reach.

Step by step: the front channel

The same flow again, this time as real HTTP. Every value below was taken from an actual run of the simulator.

1 · The client invents a secret it will not send

Before anything touches the network, the client generates a high-entropy random string and hashes it. The random string is the **code_verifier**; the hash is the **code_challenge**.

```
# 32 random bytes, base64url-encoded => exactly 43 characters
code_verifier = 3TbF_QUTB0iHthQd0rRr_6GcCTQKN_nPqc9j3P0FCBg

# BASE64URL(SHA256(ASCII(code_verifier)))
code_challenge = kPX0H7VuZIrt1ZXIk7n0C2HkA6r0YNI3lDys2XF1Crk
```

The verifier is stored in the client's own session. It must survive until step 7 and must never appear in a URL, a cookie readable by script, or a log line.

2–3 · Redirect the user to the authorization endpoint

The client does not call `/authorize` itself. It sends a `302` so that the *browser* goes there, carrying the parameters in the query string.

```
GET /authorize?response_type=code
    &client_id=demo-web-app
    &redirect_uri=http%3A%2F%2Flocalhost%3A9020%2Fcallback
    &scope=openid+profile+accounts%3Aread
    &state=cclt66TixPmn7BXxpsyNEA
    &code_challenge=kPX0H7VuZIrt1ZXIk7n0C2HkA6r0YNI3lDys2XF1Crk
    &code_challenge_method=S256

Host: localhost:9000
```

PARAMETER	WHAT IT IS FOR
response_type	Always <code>code</code> in OAuth 2.1. The other values are gone.
client_id	Which application is asking. Public, not a credential.
redirect_uri	Where to send the code. Compared by exact string match against registration.
scope	What the application wants to be allowed to do. Ask for the least you need.
state	Random, per-request. Echoed back so the client can recognise its own callback.
code_challenge	The hash from step 1. The server stores it against this request.
code_challenge_method	S256 . Send it explicitly — omitting it means <code>plain</code> , which protects nothing.

Step by step: the exchange

The user takes over, and hands back a code that is worth nothing by itself. Turning it into a token takes a direct call, and a proof.

4–6 • The user authenticates, consents, and comes back with a code

Everything in this step happens on the authorization server's own origin. The client is not running, cannot see the password, and cannot influence the consent screen. When it is done, the server redirects the browser back:

```
HTTP/1.1 302 Found
Location: http://localhost:9020/callback?code=bnBKJfGg0H4Mm_fzI4bE622-5t0w7U8kbKIHpubbQ4o
&state=cclt66TixPmn7BXxpsyNEA
&iss=http://localhost:9000
```

Two checks the client must make here. First, state has to equal the value it generated in step 2 — otherwise this callback was not started by this browser session, and the code must be dropped. Second, iss must name the authorization server the flow began with (RFC 9207), which prevents a code from one server being replayed at another.

7 • Redeem the code, and prove you started the flow

This is a server-to-server `POST`. The browser is not involved and never sees any of it.

```
POST /token HTTP/1.1
Host: localhost:9000
Content-Type: application/x-www-form-urlencoded
Authorization: Basic ZGVtbY13ZWItYXBwOndYi1hcHAAtc3VwZXItc2VjcmV0  <- only if confidential

grant_type=authorization_code
&code=bnBKJfGg0H4Mm_fzI4bE622-5t0w7U8kbKIHpubbQ4o
&redirect_uri=http://localhost:9020/callback
&code_verifier=3TbF_QUTB0iHthQd0rRr_6GcCTQKN_nPqc9j3P0FCBg
```

8 • What the authorization server checks, in order

1. Does this `code` exist, and has it been used before? A replay means someone has a copy; every token it produced is revoked.
2. Has it expired? Codes live about a minute, because they cross the browser.
3. Was it issued to *this* `client_id`? A code belongs to one client.
4. Does `redirect_uri` match the one from step 3? Both halves of the flow must agree.
5. Does `SHA256(code_verifier)` equal the stored `code_challenge`?
6. If the client is confidential, did it authenticate correctly?

Any failure returns `400 invalid_grant` and no token — and check 5 is the only one an attacker holding a stolen code cannot satisfy.

Step by step: the tokens

What comes back, how it is used, and what the failures mean.

9 · The tokens come back

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwOi8v...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "openid profile accounts:read",
  "refresh_token": "hz9ZLvVJ2akqMFGb7pQx...",
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwOi8v..."
}
```

`Cache-Control: no-store` is required, not decorative: these are credentials. Note too that `scope` comes back in the response — the user may have granted **less** than was asked for, and a correct client reads what it received.

10–11 · Call the API

```
GET /api/accounts HTTP/1.1
Host: localhost:9010
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

The token goes in the `Authorization` header — never in a query string, where it lands in every access log on the path. OAuth 2.1 removes the query-string option OAuth 2.0 allowed.

When it goes wrong: decoding the error

ERROR	USUAL CAUSE
<code>invalid_grant</code>	The code was used already, expired, issued to another client, or the <code>code_verifier</code> did not match. Deliberately vague: the server will not tell an attacker which check failed.
<code>invalid_client</code>	Wrong <code>client_id</code> or secret, or a confidential client that sent no credentials at all.
<code>invalid_request</code>	A required parameter is missing or duplicated. Commonly a forgotten <code>code_challenge_method</code> .
<code>unauthorized_client</code> <code>invalid_scope</code>	The client is not registered for this grant type, or asked for a scope it is not allowed (or that does not exist).
<code>access_denied</code>	The user pressed Deny. Not a bug; handle it gracefully.

That is the entire flow. Eleven steps, two channels, one hash comparison. Every remaining chapter is either a detail of what is inside the tokens, or a rule protecting one of these steps.

The attack PKCE prevents

The authorization code has to cross the browser, and the browser is not a private place. So assume the code leaks — and ask what it is worth.

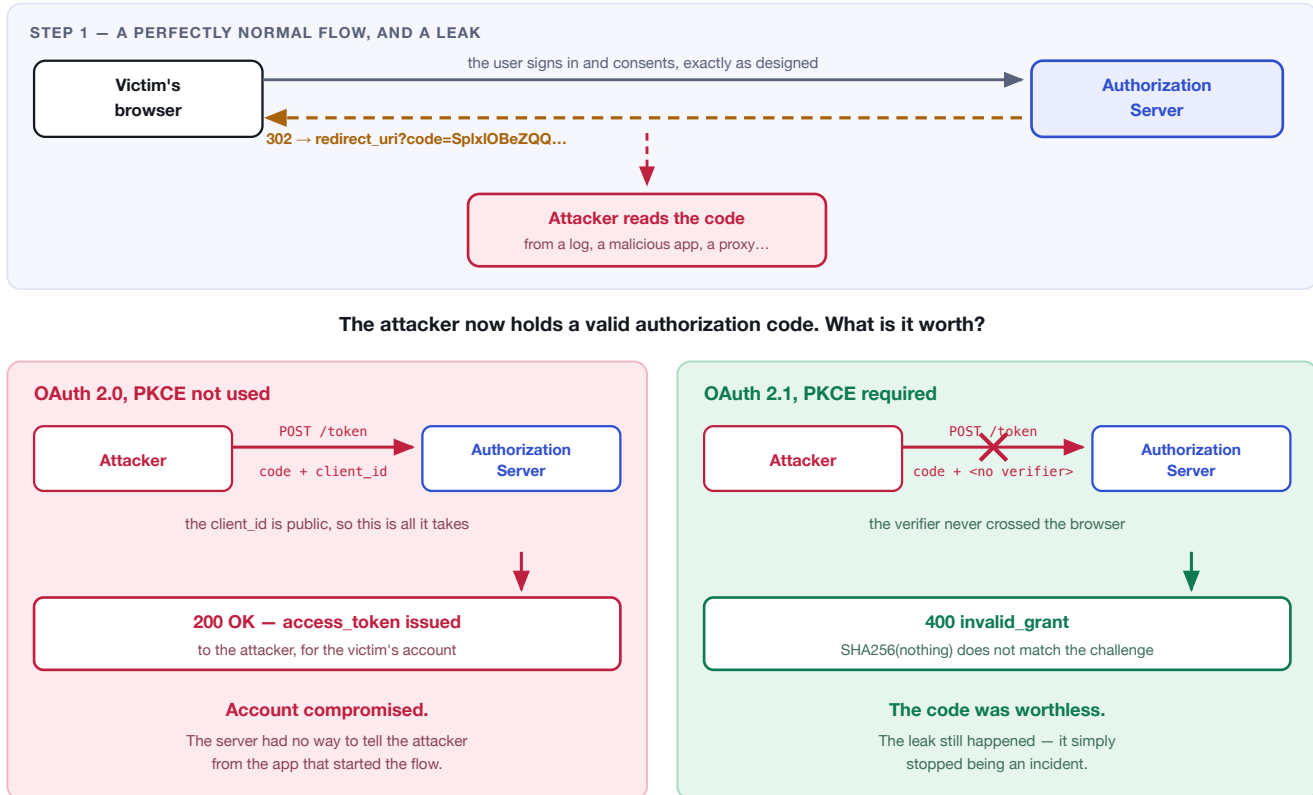


Figure 5.1 — Identical theft, two outcomes. PKCE does not prevent the code from leaking; it removes the value of having it.

How codes actually leak

None of these are exotic. Every one has produced real CVEs:

- **A malicious app on the same device** registering the same custom URI scheme (`myapp://callback`) — the OS may hand the redirect to either app.
- **Server access logs.** The callback URL, complete with code, is written to disk by every proxy and web server in the path.
- **The `Referer` header** leaking the callback URL to any third-party script or image on the landing page.
- **An open redirect** on the client's own domain, used to bounce the code to the attacker's server.
- **Browser history** on a shared or managed machine.
- **A TLS-terminating proxy** on a corporate or hostile network.

Why OAuth 2.1 requires PKCE even for confidential clients. A server-side app with a secret is already protected against most of this — the attacker would need the secret too. But PKCE also protects against a code being *injected* into an honest client's session, costs nothing, and removes any need to reason about which category a client falls into. Universal rules fail safe; conditional ones get misapplied.

PKCE, precisely

Proof Key for Code Exchange (RFC 7636). One random string, one hash, one comparison — and the authorization code stops being worth stealing.

The name is unhelpful; the idea is simple. Before starting the flow the client invents a secret, sends only a *fingerprint* of it through the browser, and later proves it holds the original. Only the app that began the flow can finish it.

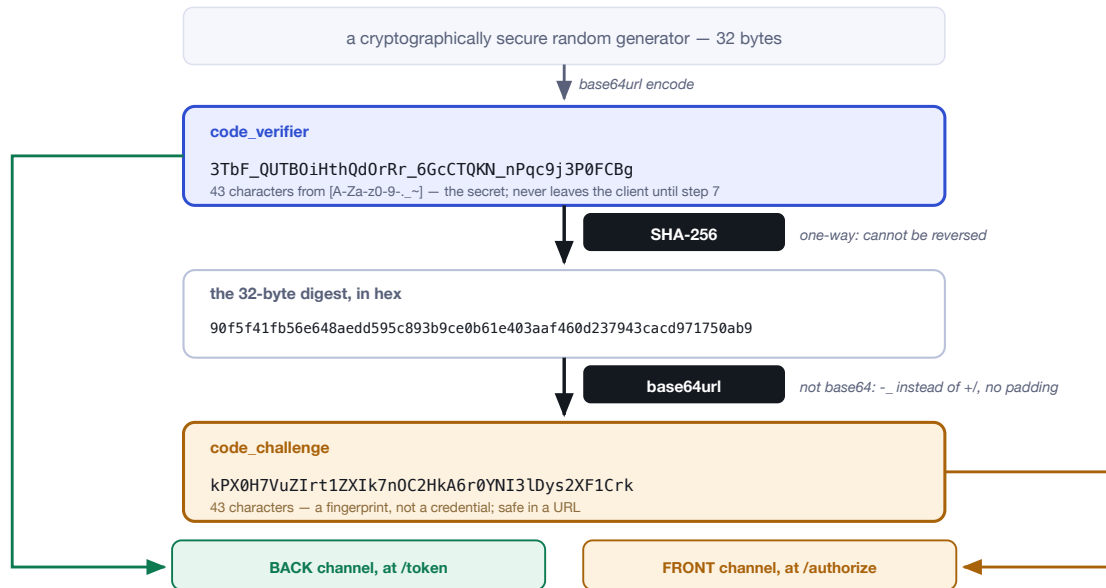


Figure 6.1 — Deriving the challenge from the verifier, with real values. The two results travel on different channels, and only one of them is a secret.

Why a hash is enough

The attacker sees the challenge

It is right there in the authorization URL. To use the stolen code they would need a string that hashes to it — that is a SHA-256 preimage, which nobody can compute. The challenge is public *by design*.

The attacker cannot guess the verifier

It is 256 bits of randomness. There is no dictionary, no pattern and no structure to attack. This is why the specification insists on a *cryptographically secure* generator, not `Math.random()`.

Never use `code_challenge_method=plain`. With `plain`, the “challenge” is the verifier, sent in the clear through the browser — so anyone who sees the authorization request can redeem the code. It exists in RFC 7636 only for devices that genuinely could not compute SHA-256 in 2015. Worse, RFC 7636 says that an **omitted** `code_challenge_method` defaults to `plain`. Always send S256 explicitly.

The check, and how to write it

What the server does with the two halves, and the code that produces them.

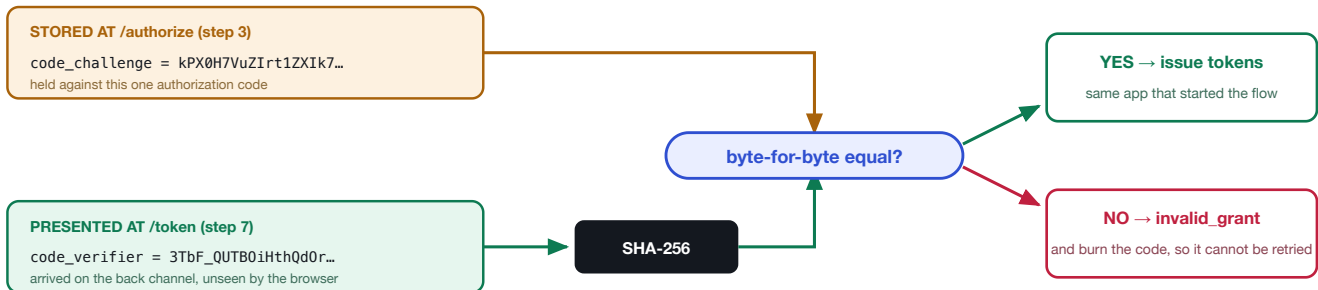


Figure 6.2 — The whole verification. Compare in **constant time**, so a rejection cannot be timed byte by byte, and treat a failure as a security event, not a retry.

Generating the pair

Browser (Web Crypto)

```

const b64url = (b) => btoa(
  String.fromCharCode(...b))
  .replace(/\+/g, '-')
  .replace(/\//g, '_')
  .replace(/=+$/, '');

const verifier = b64url(
  crypto.getRandomValues(new
    Uint8Array(32)));

const hash = await crypto.subtle.digest(
  'SHA-256',
  new TextEncoder().encode(verifier));

const challenge = b64url(new
  Uint8Array(hash));
  
```

Node.js

```

const crypto = require('node:crypto');

const verifier = crypto.randomBytes(32)
  .toString('base64url');

const challenge =
  crypto.createHash('sha256')
    .update(verifier).digest('base64url');
  
```

Python

```

import base64, hashlib, os

b64 = lambda b:
  base64.urlsafe_b64encode(b).rstrip(b'=')

verifier = b64(os.urandom(32))
challenge =
  b64(hashlib.sha256(verifier).digest())
  
```

In all three, the SHA-256 is taken over the **base64url text** of the verifier, never over the raw random bytes. Getting that backwards is the most common PKCE bug and the hardest to spot, because both values still look like valid base64url.

One pair per authorization request. A verifier is a nonce, not a configuration value: generate a fresh one every time and discard it the moment the code is exchanged. On the server side, compare with `timingSafeEqual` rather than `==`, and treat a mismatch as a security event — invalidate the code, return the generic `invalid_grant`, and never reveal which check failed. The common implementation mistakes are collected in Chapter 13.

Anatomy of a token

An access token is usually a JWT: three base64url segments joined by dots. Two of them are plain text that anyone can read.

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpzZW50L3R1b3R5bS1FYXJIZkN0.  
eyJpc3MiOiJodHRwOi8vbG9jYXRob3N0OjkwMDAiLCJzZW50L3R1b3R5bS1FYXJIZkN0.  
OkNChhb8y_TlJlTQuscjcfwn2VKkzJZPB0fzVrsgjNsesPiNaCJaZZgq5QxL
```

1 · HEADER

Which algorithm signed it, and which key (kid) to verify with.

Readable by anyone. Never trust its alg blindly.

2 · PAYLOAD (CLAIMS)

Who, what, where, until when. This is the authorization itself.

Also readable by anyone — so never put a secret in here.

3 · SIGNATURE

RSA-SHA256 over segments 1+2, made with the AS's private key.

The only part that cannot be forged or edited.

Figure 7.1 — A JWT is not encrypted. Base64url is an encoding, not a cipher: paste any token into a decoder and read it. The signature does not hide the contents — it proves nobody changed them.

What the segments say once decoded

```
// segment 1 - the header
{ "alg": "RS256", "typ": "at+jwt", "kid": "sim-EarHfCN6MZTq0Jjo" }

// segment 2 - the claims
{ "iss": "http://localhost:9000", // who issued it
  "sub": "u-1001", // whose data it acts on
  "aud": "http://localhost:9010", // which API may accept it
  "client_id": "demo-web-app", // which app is holding it
  "scope": "openid profile accounts:read", // what it may do there
  "jti": "78e3f1b14a7c70e11fac6fc1", // unique id, for revocation
  "iat": 1789228222, "nbf": 1789228222, // issued at / not before
  "exp": 1789228522 } // expires - 300 seconds later
```

Why typ: at+jwt matters

RFC 9068 gives access tokens their own type, so an API can reject anything that is not one — an ID token, for instance. Without it, two different tokens from the same issuer look interchangeable.

Opaque tokens are also fine

Plenty of servers issue random strings instead. The API then calls `/introspect` to find out what a token means. Same protocol, different trade-off — see Chapter 12.

A token is a bearer credential. Whoever holds it can use it, exactly as with a password. There is no cryptographic binding to the client that received it, which is why they are short-lived, scoped and audience-restricted — three ways of limiting the damage rather than preventing the theft.

Three tokens, three jobs

A single token response can contain three completely different things. Confusing two of them is the most common OAuth bug in production code.

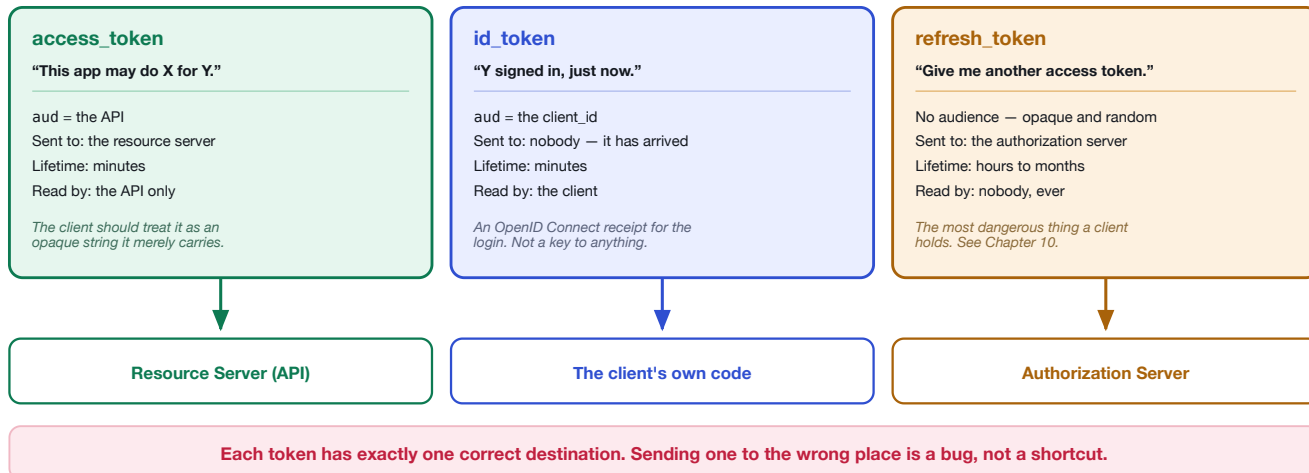


Figure 7.2 — Three tokens, three consumers. The aud claim is not documentation: it is the mechanism that enforces this separation.

The mistake, and why the API must refuse it

An ID token is a JWT from the same issuer, so it is tempting to send it as a Bearer token. It looks valid. It is not:

```
# The ID token's own claims say who it is for:
"aud": "demo-web-app"           # the CLIENT, not the API
"typ": "JWT"                    # not at+jwt
# ...and it carries no `scope` at all, so there is nothing to authorize.
GET /api/accounts Authorization: Bearer <id_token> -> 401 invalid_token
```

An API accepting an ID token is accepting a credential minted for somebody else, with no statement of what it may do — and in a federated setup that is exploitable.

The habit to build. When a call fails with 401, check which token you sent before you check anything else. “I sent the ID token” accounts for a remarkable share of lost afternoons. Lab 8 in the simulator does exactly this, twice.

Where OpenID Connect fits

OAuth alone never tells the client *who* the user is; it only hands over authority. OpenID Connect is a thin identity layer on top that adds exactly that — one scope, one extra token, one endpoint. “Sign in with Google” is OpenID Connect; “let this app read your Drive” is plain OAuth, and most deployments do both at once in the one flow from Chapter 4.

Chapter 8 covers it properly: the ID token claim by claim, the six checks a client must run on it, why `nonce` and `state` are different things, and `/userinfo`. Lab 12 runs all of it.

Opaque or self-contained?

An access token comes in two shapes, and the choice is one of the most consequential an API team makes. It decides whether your authorization server sits on the critical path of every single request.

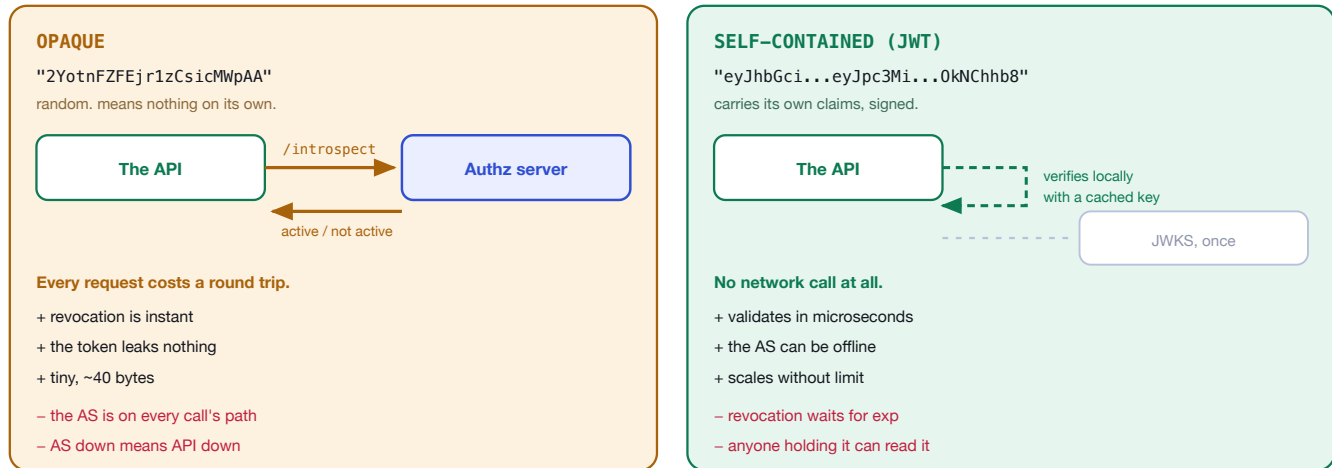


Figure 7.3 — The same authorization, two representations. Lab 10 calls one endpoint of each kind against a revoked token and prints the two different answers.

	OPAQUE	SELF-CONTAINED (JWT)
What it is	A random reference. No structure.	Signed header, payload and signature.
How it is checked	POST <code>/introspect</code> , every request.	Signature verified offline against a cached key.
Revocation	Immediate.	Not until exp.
Latency	A round trip per call.	None.
Size	Tens of bytes.	Hundreds to a few thousand.
What it reveals	Nothing at all.	Its whole payload, to anyone holding it.
If the AS is down	Your API stops working.	Your API keeps working.

Choosing between them

The honest summary: opaque tokens buy control, JWTs buy independence, and almost everyone ends up buying a bit of both.

Reach for opaque when...

- Instant revocation is a hard requirement — payments, admin consoles, anything a security team must be able to kill *now*.
- The API and the authorization server are close: same company, same network, a round trip you can afford.
- Token contents are sensitive. Roles, internal ids and email addresses in a JWT are readable by anyone who gets the token.
- The token must be small — constrained devices, tight headers, QR codes.

Reach for JWT when...

- Many services validate tokens independently — microservices, a service mesh, validation at the edge or in a gateway.
- Third parties call your API and you cannot have them, or your AS, absorbing an introspection call per request.
- Throughput is high enough that a round trip per call is a real cost.
- The authorization server must not be a single point of failure for every API in the estate.

What most people actually do

The common production answer is a deliberate compromise, and it is the one this book has been arguing for since Chapter 9: **self-contained access tokens with deliberately short lifetimes**. You get offline validation, and you bound the revocation gap to the token's lifetime rather than eliminating it.

- **Five to fifteen minutes** is the usual window. Long enough to avoid hammering the token endpoint, short enough that a revoked token dies quickly.
- **Refresh tokens carry the real revocation power**. They are almost always opaque and checked against a database on every use — so revoking a session stops the *next* refresh even though the current access token lives out its few remaining minutes.
- **Use introspection selectively**. Keep JWT validation for ordinary reads and introspect only on the operations that genuinely cannot tolerate a stale answer. Lab 10's two endpoints are exactly this pattern.

A rule for clients, regardless of shape. Never parse an access token. Even when it is obviously a JWT, its format is a private contract between the authorization server and the API — either may switch to opaque tomorrow without telling you, and nothing in the specification promises you can read it. If your client needs to know who the user is, that is what the **ID token** is for (Chapter 8). If it needs to know whether a token is still good, it should try the call and handle 401.

The other two tokens have no choice to make. An **ID token is always a JWT** — it exists to be read and verified by the client, so it could not be anything else. A **refresh token is almost always opaque**, because it is redeemed only at the authorization server, which can simply look it up.

OpenID Connect: who is this?

OAuth hands an application authority. It never says whose. If you need to know *who just signed in*, you need the thin identity layer that sits on top of it — and you need to stop reaching for the access token.

The shortcut that does not work

The tempting move is to treat a successful token request as proof of identity: the user approved something, so they must be here. That reasoning fails in three separate ways.

- **An access token is not addressed to you.** Its `aud` names the API. Nothing in it says which client obtained it, so a token your app received from somewhere else looks identical to one it earned.
- **It says nothing about *when* anyone authenticated.** A token minted from a refresh token six weeks later looks exactly like one from a fresh login.
- **It may be opaque.** You are not entitled to read an access token at all — it is the API's to interpret, not yours.

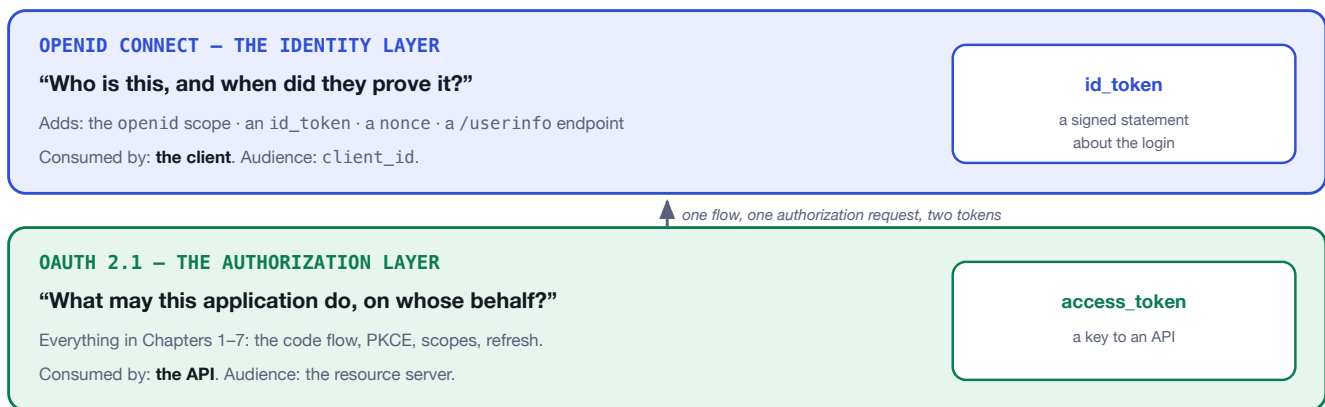


Figure 8.1 — OpenID Connect is not a different protocol or a different flow. It is the same Authorization Code request from Chapter 4 with one extra scope, returning one extra token addressed to a different party.

The whole of it, in one line. Ask for the `openid` scope and the token response gains an `id_token`: a JWT whose `aud` is **your `client_id`**, saying who signed in and when. That is the difference between “let this app read your Drive” and “sign in with Google”.

The ID token, claim by claim

A real one, from Lab 11. Every field is doing a job, and two of them are the ones people skip.

```
{ "iss":      "http://localhost:7000",    // who issued it
  "sub":      "u-1001",                  // WHO SIGNED IN – the identifier
  "aud":      "demo-web-app",            // the CLIENT, not the API
  "iat":      1789331908,                // when the token was minted
  "exp":      1789332208,                // when it stops being usable
  "auth_time": 1789331908,                // when they actually authenticated
  "nonce":    "4xxCDNzp2nXsz22J",        // ties it to THIS request
  "name":     "Alice Liddell",            // from the `profile` scope
  "email":    "alice@lab.test" }          // from the `email` scope
```

The two that matter most

CLAIM	WHAT IT IS FOR
sub	The identifier for the user, and the only one you should key on. It is stable and unique <i>within this issuer</i> — so your database key is the pair (iss, sub), never sub alone if you accept more than one issuer.
nonce	A random value you generate, send on the authorization request, and compare when the ID token comes back. It proves the token was minted for <i>this</i> login rather than replayed from another one.
auth_time	When the user actually authenticated — which may be long before this token was issued. Check it if an operation needs a <i>fresh</i> login; ask for it with max_age.
azp	Present when the token has several audiences: names which client it was issued to. Check it whenever aud is an array.

Never key your user records on email. Email addresses change, get reassigned by employers, and arrive unverified unless email_verified says otherwise. A user who changes their address would become a stranger to your system; worse, a reassigned address could make a stranger into that user. Store sub; treat email as a display attribute that can change under you.

Validating an ID token

Chapter 12 lists the four checks an *API* runs on an access token. These are the checks a *client* runs on an ID token. They are not the same list, and the last two are the ones that get skipped.

1. **Verify the signature** against the issuer's published key, selected by the `kid` in the header. Pin the algorithm in your code — never trust the token's own `alg`.
2. **Check `iss`** is exactly the issuer you expect. String equality, no prefix matching.
3. **Check `aud` contains your `client_id`**. If it is an array, also check `azp` names you. This is what stops another client of the same issuer replaying its ID token at you.
4. **Check `exp`** has not passed, allowing a little clock skew.
5. **Check `nonce`** equals the value you generated for this authorization request. Without it, a token captured from an earlier login can be fed back to you.
6. **Check `auth_time`** if you asked for `max_age`, or if this operation needs a recent authentication rather than an old session.

```
// Lab 11 does exactly this. Checks 1-4 in one pass:
const v = C.verifyJwt(idToken, {
  getKey: () => keyFromJwks,      // signature, by kid
  issuer: 'http://localhost:7000', // iss
  audience: 'demo-web-app',       // aud == our client_id
  requiredTyp: 'JWT' });          // exp is checked inside

// and the one no library can do for you, because only you know it:
if (claims.nonce !== theNonceWeGenerated) reject();
```

nonce and state are not the same thing

They look alike — both random, both sent out and checked on return — but they protect different things, and you need both.

	TRAVELS	PROTECTS
state	On the authorization request, echoed on the redirect .	The client's flow . Proves this callback belongs to a request this browser session started, and carries your own UI state across the redirect.
nonce	On the authorization request, echoed inside the ID token .	The token itself . Proves this ID token was minted for this login, and is not one replayed from a different session.

A useful way to remember it: `state` is checked before you have a token; `nonce` is checked after you open one. Lab 11 verifies both, and Lab 11 step 4 shows an ID token addressed to a different client being refused.

Claims, /userinfo, and what this is not

Where the user's details come from, which scope produces which claim, and the three things OpenID Connect is routinely mistaken for.

Two places to get claims

In the ID token

Arrives with the token response, already signed. **No extra request.** A snapshot from the moment of login, so it can go stale. Best for what you need immediately: a display name, an avatar.

From /userinfo

Fetches with the **access token** — not the ID token. Always current, and keeps a large profile out of every token. Best when claims change, or when you need them later.

Which scope produces which claim

SCOPE	YOU GET
openid	Required. Without it there is no <code>id_token</code> and <code>/userinfo</code> refuses you. This single scope is the opt-in to OpenID Connect.
profile	<code>name</code> , <code>preferred_username</code> , and the other display fields.
email	<code>email</code> and <code>email_verified</code> — check the second before trusting the first.
offline_access	A refresh token, so you can act later without the user present.

```
# Lab 11, step 5 - note which token it takes
GET /userinfo Authorization: Bearer <the ACCESS token>

-> 200 {"sub":"u-1001","name":"Alice Liddell",
      "preferred_username":"alice","email_verified":true}
```

Three things it is not

- **Not a session.** The ID token proves a login happened; it is not a session cookie and its `exp` is not your session length. Validate it once, then issue your own session and forget the token.
- **Not authorization.** An ID token says who, never what. Do not send it to an API and do not put permissions in it — that is what scopes and the access token are for.
- **Not a way to skip PKCE.** Everything in Chapters 4–6 still applies, unchanged. OpenID Connect rides the same flow; it does not soften a single requirement of it.

Run it. `node client.js oidc` walks all of this: the extra scope, both tokens side by side, the six validation checks, a wrong-audience token refused, `/userinfo`, and finally what happens when you drop `openid` and the identity layer disappears.

Scopes and consent

A scope is a label for a slice of authority. What an application asks for and what it gets are two different things, and it is narrowed three separate times.

1. WHAT THE APP ASKS FOR

openid profile email accounts:read payments:write

5 scopes

↓ capped by what the client is registered for

2. WHAT THE CLIENT MAY EVER REQUEST

openid profile email accounts:read payments:write

unchanged — all 5 are registered

↓ capped by the user's own entitlements — here, bob

3. WHAT THIS USER CAN DELEGATE AT ALL

openid profile email accounts:read

bob cannot grant this

payments:write

↓ narrowed by what the user actually ticks

4. WHAT THE USER APPROVES ON THE CONSENT SCREEN

openid profile accounts:read

email unticked

The token comes back with 3 scopes, not 5 — and the client must cope with that.

Figure 9.1 — Scope only ever narrows. Each stage can remove, never add, which is why the token response repeats the scope it actually granted.

Designing scopes people can consent to

Good

- `accounts:read`, `payments:write` — resource plus verb
- Separate read from write, always
- Few enough to fit on one screen
- Named for what the *user* loses if abused

Bad

- `admin`, `full_access` — unconsentable
- One scope per API endpoint — unreadable
- Scopes encoding record ids (`account:4410`)
- Internal role names leaked into the UI

Scope is not the same as permission

This distinction causes real vulnerabilities. A scope says *this application may perform this kind of operation on this user's behalf*. It says nothing about which records that user may touch.

```
# The token carries scope=accounts:read and sub=u-1001.
GET /api/accounts/ACC-7100      # <- an account belonging to u-1002

# A scope check alone would ALLOW this. The API must also check ownership.
```

The simulator's API takes the user from the token's `sub` and never from the request, which is the habit worth copying. An object reference supplied by the caller must always be authorized against the token's subject.

Refresh tokens and rotation

Access tokens expire in minutes. Users expect to stay signed in for weeks. The refresh token bridges that gap — and needs careful handling because of it.

An access token cannot be recalled once issued, so it is given a short life: a leak is a problem for five minutes, not five months. The refresh token mints replacements without troubling the user.

ONE TOKEN FAMILY, OVER TIME

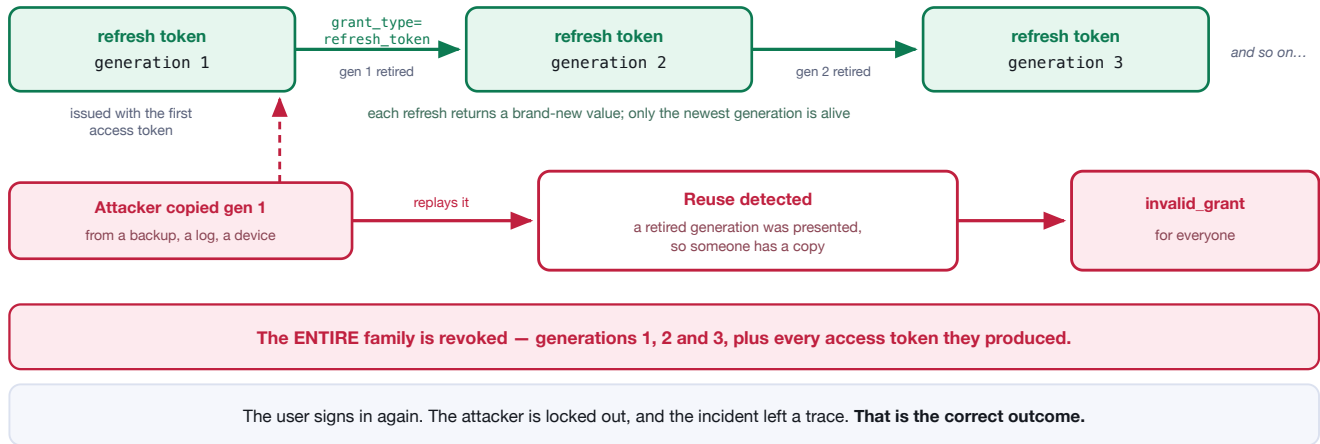


Figure 10.1 — Rotation with reuse detection (RFC 9700 §4.14). The server cannot tell thief from victim, so it trusts neither: a silent, permanent compromise becomes a visible, recoverable one.

What this buys you

- **A stolen refresh token stops working.** Whichever party refreshes second is refused, and the theft is detected rather than lasting for months.
- **The theft becomes observable.** A reuse event is a high-quality security signal, because correct clients never replay a retired token. This is why OAuth 2.1 requires rotation (or sender-constraining) for public clients.

How long should each one live?

CREDENTIAL	TYPICAL	WHY
authorization code	30–60 s	It crosses the browser and is redeemed immediately. Anything longer is pure exposure.
access token	5–15 min	Cannot be recalled, so its lifetime is the exposure window.
refresh token	hours–months	Long, but rotating and revocable. Shorter for public clients.
IdP session	hours	Decides whether a <i>new</i> authorization needs a fresh login.

The operational catch. If a client crashes after sending a refresh but before storing the new token, it will replay the old one and get the whole family revoked — logging the user out for no good reason. Persist the rotated token before you use the access token that came with it, and serialise refreshes so two tabs cannot race.

What OAuth 2.1 changed

OAuth 2.1 invents almost nothing. It deletes what proved dangerous and makes the best existing practice mandatory — and that is precisely why it is an improvement.

OAuth 2.0 is a framework of options. A decade of deployments showed which options were traps, and the security Best Current Practice (RFC 9700) documented them. OAuth 2.1 consolidates that guidance into the core, so that the default reading of the specification is now the safe one.

OAUTH 2.0 (2012)	OAUTH 2.1	WHY IT CHANGED
PKCE optional, recommended for mobile	Required for every authorization code request	Codes leak in ways nobody predicted, the defence costs almost nothing, and a universal rule cannot be misapplied.
Implicit grant (<code>response_type=token</code>)	Removed	It put an access token in the URL fragment: browser history, <code>Referer</code> headers, any script on the page — and the client could not be authenticated at all.
Password grant (<code>grant_type=password</code>)	Removed	The application handled the real password, so there was no consent, no MFA, no federation, and no way for a user to tell a real app from a phishing one.
<code>redirect_uri</code> matching left to the server	Exact string comparison	Wildcards and prefix rules let an attacker append a path and have codes delivered to themselves.
Bearer tokens allowed in query strings	Header only	Query strings are written to every access log, proxy log and browser history on the path.
Refresh tokens static and long-lived	Rotate, or bind to the client	A stolen static refresh token is permanent, undetectable access. Rotation makes the theft visible and recoverable.
Little guidance on mixing up servers	iss in the authorization response	Prevents a mix-up attack where a code from a malicious server is replayed at an honest one.

If you already know OAuth 2.0

You know OAuth 2.1. Add PKCE everywhere, stop using implicit and password, match redirect URIs exactly, and rotate refresh tokens. That is the entire migration.

On the specification's status

OAuth 2.1 is an IETF draft that consolidates published, stable RFCs. Nothing in it is speculative — every requirement already exists as a standard somewhere. Implementing it is not a bet on a draft.

The pattern worth noticing. Every deletion removes a *choice* a developer could get wrong. The most reliable security improvement is not a new mechanism but the removal of an unsafe option — which is the actual lesson of OAuth 2.1.

How an API validates a token

The resource server is the part most tutorials skip. On every single request it asks four questions, in this order.

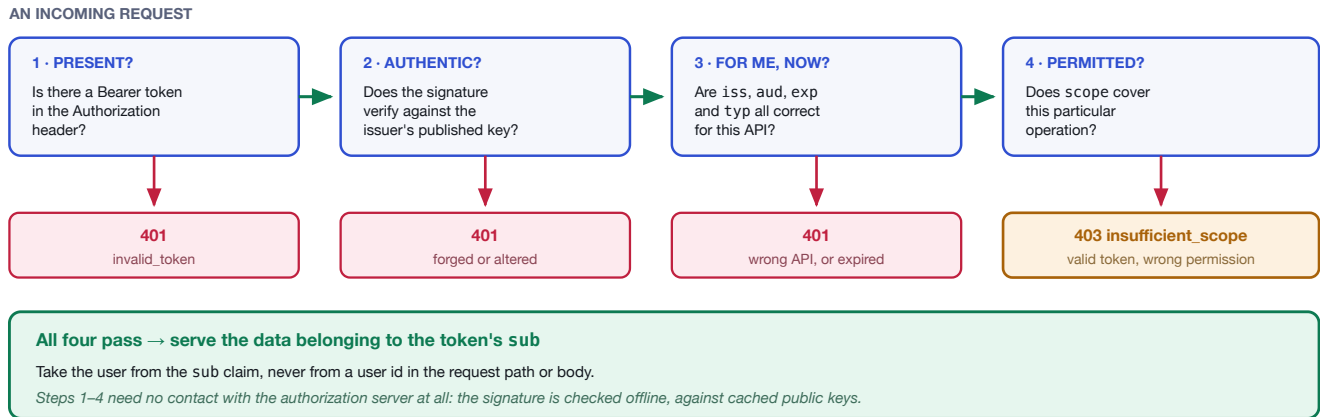


Figure 12.1 — The four gates. Note the status codes: **401** means “this token is not acceptable”; **403** means “the token is fine but does not authorize this”. Returning 401 for a scope failure sends clients into pointless refresh loops.

Local validation or introspection?

Verify the JWT locally

Fetch the issuer's public keys once from `/jwks.json`, cache them by `kid`, verify offline.

Fast and needs no network. **But** it cannot see a revocation: a revoked token stays acceptable until `exp`.

Ask the authorization server

Call `/introspect` (RFC 7662) per request. Works with opaque tokens too.

Always current, including revocation. **But** it adds a round trip to every call and makes the AS a hard dependency.

This trade-off is the real reason access tokens are short-lived: a five-minute window is the compromise between not calling the AS on every request and not honouring a revocation for too long. Lab 13 in the simulator revokes a token and then calls both kinds of endpoint, so you can watch one accept it and the other refuse.

Three ways to get validation wrong

- **Trusting the token's own `alg` header.** If you let the token choose its algorithm, an attacker sends `alg: none` (no signature at all) or swaps RS256 for HS256 and signs with your *public* key as the HMAC secret. Pin the expected algorithm in your code and reject everything else.
- **Skipping the `aud` check.** Then any token from that issuer works at your API, including one minted for a low-value service that has no business here. A valid signature only says who issued a token, not who it was for.
- **Caching keys without handling rotation.** Cache by `kid`, refetch when an unknown `kid` appears, and rate-limit that refetch. An issuer that rotates keys should not take your API down — nor should an unknown `kid` let an attacker trigger unbounded fetches.

The mistakes that matter

A checklist drawn from the failures that actually recur.

Writing a client

MISTAKE	CONSEQUENCE
Hashing the raw random bytes instead of the base64url verifier text	Every exchange fails with <code>invalid_grant</code> and nothing in the error says why.
Using base64 instead of base64url	<code>+</code> , <code>/</code> and <code>=</code> get percent-encoded in the URL, and the comparison fails.
Reusing one <code>code_verifier</code> for every request	A single leak compromises every future flow. It is a nonce: generate a fresh pair each time.
Keeping the verifier or tokens in <code>localStorage</code>	Readable by any script on the origin, so one XSS takes everything. Keep them server-side, or in memory.
Omitting <code>code_challenge_method</code>	RFC 7636 defaults it to <code>plain</code> : a strict server rejects you, a lenient one silently drops your protection.
Assuming the granted scope equals the requested scope	Features break confusingly when a user declines one checkbox. Read <code>scope</code> from the response.
Sending the <code>id_token</code> to the API	<code>401</code> , and an afternoon lost. It is a login receipt, not an API key.

Running a server or an API

MISTAKE	CONSEQUENCE
Accepting the token's own <code>alg</code> header	Opens <code>alg: none</code> and RS256→HS256 confusion. Pin the algorithm server-side.
Not validating <code>aud</code>	A token minted for any client of that issuer works at your API.
Prefix or wildcard <code>redirect_uri</code> matching	An attacker appends a path and has your users' codes delivered to them.
Letting an authorization code be redeemed twice	Whoever holds a copy can keep minting tokens. Single-use, and revoke on replay.
Returning <code>401</code> for a scope failure	Clients refresh in a loop trying to fix something a refresh cannot fix. Use <code>403</code> .
Taking the user id from the request instead of <code>sub</code>	A textbook IDOR sitting on top of otherwise correct OAuth.

A useful instinct. When OAuth surprises you, ask *which channel is this on, and who can see it?* Nearly every mistake above is a value that ended up on the wrong channel, or a check skipped because the happy path worked without it.

Run it yourself

Reading about a protocol gets you halfway. Part II of this book is a complete OAuth 2.1 deployment in four files — an authorization server, an API and a client — with fifteen labs that make each chapter happen on your own machine.

Everything you need is in this book

The four listings in Part II are the whole program: 1,933 lines, no elisions, no repository to clone, and **no dependencies to install**. They run on Node's own `node:http` and `node:crypto`, which is the point — a library would hide exactly the parts this book is about.

```
# three terminals; the first two narrate what the protocol is doing
node authz-server.js      # port 7000 - the AS and IdP
node api-server.js        # port 7010 - the protected API
node client.js happy      # Lab 1 - the correct flow, end to end
```

Each chapter, as a lab

LAB	COMMAND	WHAT IT PROVES	CHAPTER
1	<code>client.js happy</code>	The whole flow, then a call the token is not scoped for	3–4
2	<code>client.js pkce</code>	The verifier, the hash, the challenge, and the comparison	6
3	<code>client.js attack</code>	A stolen code is worthless without the verifier	5–6
4	<code>client.js nopkce</code>	PKCE off: the same theft succeeds	5–6
5	<code>client.js replay</code>	Codes are single-use, and a replay revokes what it made	4
6	<code>client.js refresh</code>	Rotation, then reuse detection killing the family	10
7	<code>client.js scope</code>	Scope narrowing, and <code>403 insufficient_scope</code>	9
8	<code>client.js idtoken</code>	ID-token and wrong-audience tokens both refused	7, 12
9	<code>client.js m2m</code>	<code>client_credentials</code> : no user, no consent	2
10	<code>client.js revoke</code>	Local validation still accepts a revoked token	12
11	<code>client.js oidc</code>	ID token, nonce, and <code>/userinfo</code>	8
12	<code>client.js agent-device</code>	Device grant: an agent with no browser	15
13	<code>client.js agent-rar</code>	Limits a scope cannot express	15
14	<code>client.js agent-exchange</code>	Delegation chains and the <code>act</code> claim	15
15	<code>client.js agent-deputy</code>	Escalation refused twice, independently	15

The command worth running twice. Lab 3 steals an authorization code and is refused. Lab 4 sets `require_pkce` to false on the running server and steals the same way — and walks off with a working access token for the victim's account. Identical theft, opposite outcomes, one flag apart. Every safety rule in `authz-server.js` can be switched like that: `curl localhost:7000/policy` lists them.

OAuth 2.1 for agentic AI

An agent is a client that acts when nobody is watching. Every assumption in Chapters 3 and 4 bends here — but the protocol needs no replacing, only three extensions and stricter administration.

What actually changes

- **There is no browser.** A redirect has nowhere to land, so the authorization code flow cannot start in the usual way.
- **The human is absent at the moment of the call.** Consent happens once, minutes or weeks before the action it authorizes.
- **One authorization fans out into many calls.** An agent plans, retries and chains requests, often across several services.
- **Scopes stop being expressive enough.** “May move money” is a reasonable thing to tell a human at a consent screen and a dangerous thing to hand an autonomous process.

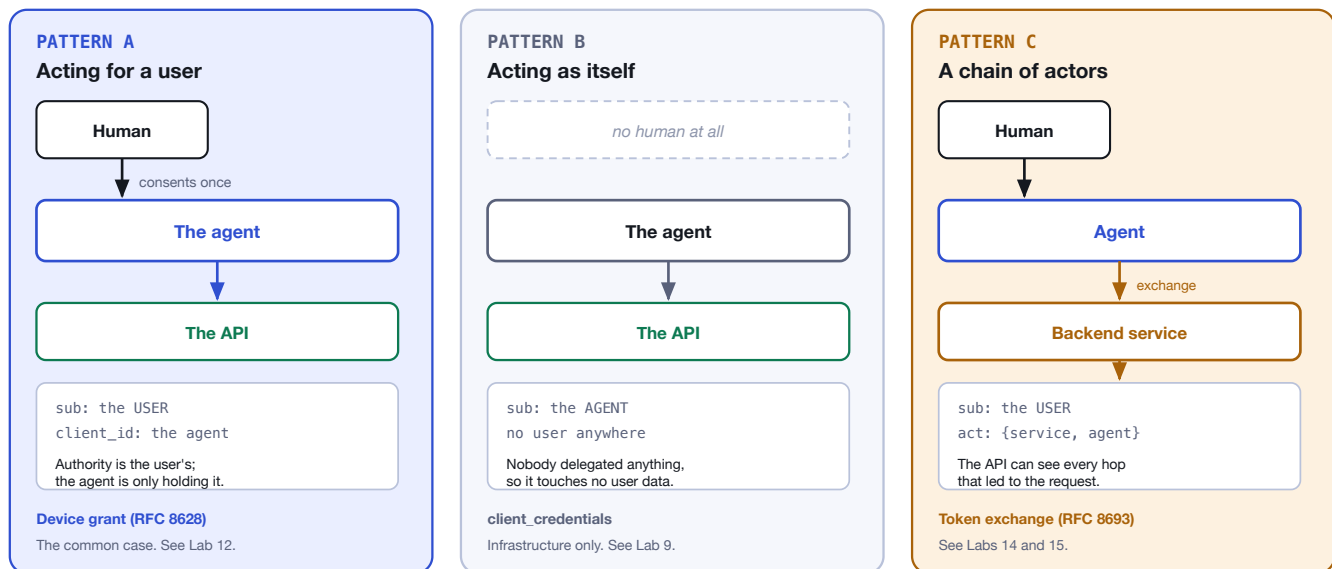


Figure 15.1 — The three patterns. Choosing the wrong one is the root of most agent authorization incidents.

The mistake to refuse, first and loudest. An agent reaching user data with `client_credentials` is not OAuth. Nobody consented, the token names no user, and your audit log can never answer “on whose behalf?” If a team asks for this because “the agent needs access to everything”, the answer is Pattern A or C.

Pattern B, and when it is actually right

It takes a paragraph rather than a page because the only hard part is knowing when it applies. **It is correct when the agent touches nobody's data** — rotating certificates, reindexing a catalogue, pruning logs. There is no resource owner, so nothing to delegate and nobody to consent: `client_credentials` is right, and Pattern A would be theatre.

The test that settles it. Ask *whose* data the agent touches. If the answer names a person, Pattern B is wrong however convenient it looks; if it is “the company's own”, B is right and the rest of this chapter mostly does not apply. **Lab 9** shows both halves: a token issued with no browser and no consent, sub set to the client id, and `/api/me` returning 404 — no user behind it.

Pattern A: an agent with no browser

The device authorization grant (RFC 8628) was written for televisions. It turns out to be exactly what an autonomous agent needs: the agent gets a short code, and the human approves it somewhere else entirely.

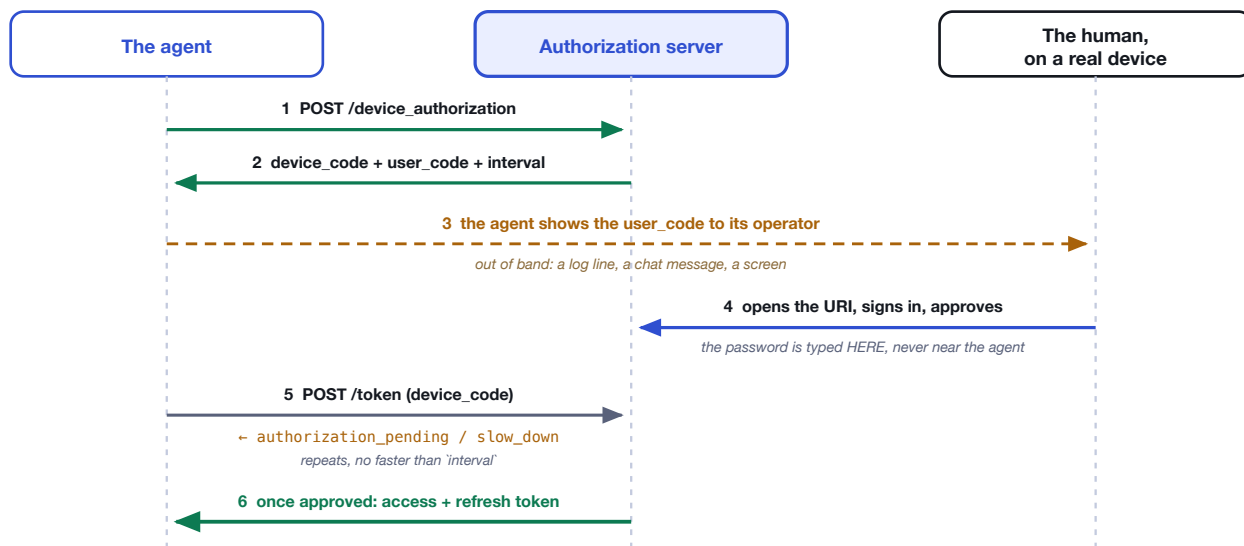


Figure 15.2 — The agent never handles a credential and never needs a redirect URI. The only thing it shows the operator is a short code that is useless to anyone who cannot also authenticate.

The exchange, as HTTP

```

POST /device_authorization
client_id=demo-agent&scope=openid accounts:read payments:write

-> 200
{ "device_code": "GmRh...", "user_code": "PCRF-GFSB",
  "verification_uri": "http://localhost:7000/device",
  "expires_in": 300, "interval": 5 }

# then, repeatedly, until the human acts:
POST /token
grant_type=urn:ietf:params:oauth:grant-type:device_code&device_code=GmRh...

-> 400 {"error":"authorization_pending"} # keep waiting
-> 400 {"error":"slow_down"}           # you polled too fast
-> 200 {"access_token":"eyJ...", "refresh_token":"..."}
  
```

Four things the AS administrator owns here. (1) The user_code alphabet — drop characters people confuse, and keep enough entropy that it cannot be guessed inside its short life. (2) A short expires_in; five minutes is generous. (3) Enforce interval server-side and return slow_down — otherwise one agent can hammer the token endpoint while it waits. (4) Make the approval screen say plainly that the approver is authorizing *an autonomous program*, not a login.

Pattern C: who is acting, on whose behalf

An agent rarely calls the API directly. It calls a service, which calls another. Each hop must carry the user’s authority without impersonating the user — and without quietly acquiring authority of its own.

The wrong answers, first

Passing the user’s token onward

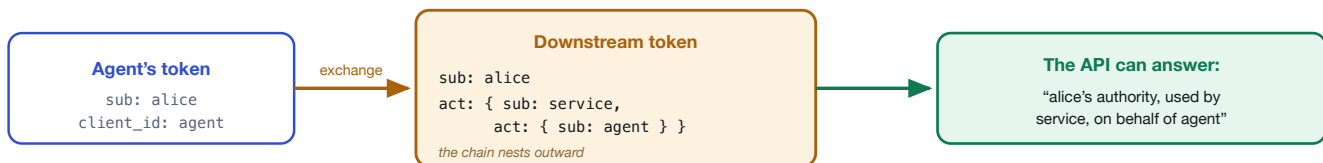
Every downstream service now holds a credential that works everywhere the user does, for as long as it lives. One compromised service becomes total compromise, and the audit log shows only “alice” at every hop.

Each service using its own machine token

The user disappears entirely. The API sees a service acting as itself and cannot tell an authorized request from a bug or an attacker who reached that service.

Token exchange, and the act claim

RFC 8693 lets a service hand in the token it received and get back one scoped for the next hop. The `sub` stays the user; a new `act` claim records who is acting, nested outward.



Scope may only ever **narrow** across an exchange. You cannot exchange your way up.

Figure 15.3 — Lab 14 builds this chain and prints it; Lab 15 tries to widen it and is refused twice.

```

POST /token
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&subject_token=<the token the service received>
&subject_token_type=urn:ietf:params:oauth:token-type:access_token
&scope=accounts:read # same or less. never more.
&resource=https://api.example.com # bind it to one audience
  
```

When a scope is not specific enough

A scope is a label fixed at registration time. It cannot say “up to \$100, from this account”. For a human clicking one button that is usually fine; for a process acting a thousand times unattended, it is not.

What a scope can say

```
scope=payments:write
```

“This agent may move money.” Any amount, from any of the user’s accounts, to anywhere, for as long as the token lives. The API has nothing finer to check.

What authorization_details can say

```
[{ "type": "payment_initiation",
  "actions": ["initiate"],
  "maxAmount": "100.00",
  "fromAccount": "ACC-4410" }]
```

The same permission, bounded. The consent screen can show it, the token carries it, and the API enforces it.

How it flows

RFC 9396 adds one JSON parameter, `authorization_details`. In the lab it is accepted at `/device_authorization`, approved by the human, embedded in the token as a claim, and checked by the API on every payment.

```
# 1. the agent asks for bounded authority, not a broad scope
POST /device_authorization
  scope=payments:write
  &authorization_details=[{"type":"payment_initiation","maxAmount":"100.00",
                           "fromAccount":"ACC-4410"}]

# 2. it ends up inside the access token, alongside the scope
{ "sub": "u-1001", "client_id": "demo-agent", "scope": "payments:write",
  "authorization_details": [ { "type": "payment_initiation", ... } ] }

# 3. the API treats it as binding, not advisory
amount 50   from ACC-4410  -> 201 created
amount 5000 from ACC-4410  -> 403 insufficient_authorization
amount 10   from ACC-9982  -> 403 insufficient_authorization
```

Note what happened in that last block. The scope `payments:write` was satisfied on all three requests. Two were refused anyway. That is the entire argument for structured grants: the coarse permission stays true while the specific limit does the work. Lab 13 runs exactly these three calls.

Administering it

- **Publish the types you accept** as `authorization_details_types_supported`, and reject unknown types rather than ignoring them — silently dropping a limit is worse.
- **Render them in plain language at consent.** A structured grant the user cannot read is not consent.
- **Be consistent on a mismatch:** `403` with a body naming the granted limit, so a caller can tell “not allowed” from “not allowed *this much*”.

Running the authorization server

Everything an administrator decides once, that every agent then lives inside. These are the settings that matter, in the order they bite.

DECISION	WHAT TO DO, AND WHY
One client per agent	Never share a <code>client_id</code> across agents or deployments. It is the only handle you have for revoking, rate-limiting or attributing one agent without affecting the rest.
Grants, narrowly	Register each agent for the one grant it needs. An agent that only ever uses the device grant must not be able to use <code>client_credentials</code> — that is the escape hatch into Pattern B.
Short access tokens	Minutes, as for any client. Agents make this easier, not harder: they refresh without a human present.
Rotate refresh tokens	Mandatory here. An agent's refresh token is a long-lived credential sitting in a process that reads untrusted input all day. Rotation plus reuse detection is what makes a leak survivable.
Consent that names the agent	The screen must say which agent, what it may do, for how long, and that it will act unattended. “Sign in with...” wording is wrong here — nobody is signing in.
Structured grants	Offer <code>authorization_details</code> for anything with a magnitude: money, deletion, sending on the user's behalf. Publish the types you support.
Audience binding	Require <code>resource</code> (RFC 8707) so a token minted for one API cannot be replayed at another. An agent touching five services should hold five narrow tokens, not one wide one.
A kill switch	Revocation by <code>client_id</code> , not just by token. When an agent misbehaves at 3am you want one call that stops all of it, including every refresh-token family it holds.
Step-up for the sharp edges	Some actions should require a fresh human approval regardless of the token: re-run the device grant for that specific operation rather than pre-granting it.

On DPoP (RFC 9449). Everything above still assumes **bearer** tokens: whoever holds one can use it. Sender-constrained tokens bind a token to a key the client proves it holds, so a stolen token is inert. For agents — long-lived, widely deployed, handling untrusted input — this is the most valuable upgrade available. It is not implemented in the labs; treat it as the next thing to read after this book.

Running the resource server

The API is where an agent’s authority becomes an action. The four checks of Chapter 12 all still apply — these are the ones agents add.

Five additional checks

- 1. **Read the actor chain, and log it.** If the token carries `act`, the request came through one or more services. Record the whole chain next to the action, or you will never reconstruct an incident.
- 2. **Enforce `authorization_details` as binding.** If the token carries a structured grant covering this operation, the request must fit inside it. A scope check alone is not sufficient.
- 3. **Never accept an identity from the request body.** Always take the user from `sub`. Agents assemble requests from model output; a caller-supplied account id is exactly the value that will be wrong.
- 4. **Make writes idempotent.** Agents retry — on timeouts, on `slow_down`, on their own planning loops. Require an idempotency key on anything that moves money or sends a message.
- 5. **Rate-limit and profile per `client_id`.** A human makes a payment a minute at most. An agent can make a thousand. Alert on the shape of the traffic, not just its volume.

Anti-patterns, and what to do instead

INSTEAD OF...	DO THIS
A long-lived API key in the agent’s config	Device grant plus rotating refresh tokens. Same convenience, revocable, and attributable to a user.
<code>client_credentials</code> to reach user data	Pattern A or C. If no human ever consented, the agent has no business with that data.
One shared <code>client_id</code> for the whole fleet	One per agent. Revocation and attribution both depend on it.
A broad scope “so it does not break”	Narrow scope plus <code>authorization_details</code> for the magnitude.
Forwarding the user’s token between services	Token exchange, so each hop is scoped, audience-bound and recorded in <code>act</code> .
Putting tokens anywhere a model can read them	Keep credentials out of prompts, context windows and tool output. A token in a context window is a token in every log you own.

The summary, if you keep one page. Agents do not need a new protocol. They need the flow that works without a browser, a way to record who acted, limits a scope cannot express, and an administrator who registered each one separately and can switch it off. Labs 12–15 build all four.

The Labs

Everything from here on is runnable. The four listings in this part are complete — nothing is elided, nothing is downloaded. Type or copy them into four files and you have a working OAuth 2.1 authorization server, a protected API, and a client that drives ten experiments against them.

What you need

Node.js 18 or newer, and nothing else. There is no `package.json`, no `npm install` and no dependency to audit: every line runs on Node's own `node:http` and `node:crypto`. That is deliberate. A dependency would hide exactly the parts of the protocol this book is about.

The four files

FILE	LINES	LISTING	WHAT IT IS
<code>crypto-lab.js</code>	126	A	PKCE and JWT primitives. Required by the other three.
<code>authz-server.js</code>	850	B	The authorization server and identity provider, on port 7000.
<code>api-server.js</code>	252	C	The resource server, on port 7010.
<code>client.js</code>	705	D	The client, and all fifteen labs.

Put all four in one directory — the three servers do `require('./crypto-lab')`, so they expect it alongside them.

Starting it up

Three terminals. The first two stay running and print what the protocol is doing; the third is where you work.

```
# terminal 1 – the authorization server and IdP
node authz-server.js

# terminal 2 – the API
node api-server.js

# terminal 3 – the labs
node client.js list
node client.js happy
```

The client needs no browser. It walks the front channel with a cookie jar, following redirects and filling in the login and consent forms itself, so each lab is one command whose result is printed rather than clicked. You can still open `http://localhost:7000/authorize?...` by hand if you want to see the screens a real user would.

Keep terminal 1 in view. The authorization server narrates every decision it makes — which rule it checked, what it compared, and why the request passed or failed. Reading a chapter here and then watching the matching line appear there is the fastest way to make any of this stick.

Labs 1–3: the flow, and the proof

Start here. The first three labs build the correct flow, take the PKCE transform apart, and then try to steal from it.

LAB 1

The correct flow, end to end

```
node client.js happy
```

Every step of Chapter 4 in order: the PKCE pair, the authorization request, login, consent, the code, the token exchange, the decoded access token, then a call to the API and a call the token is *not* scoped for.

PASS state matches – this callback is ours

PASS tokens issued

PASS 200 – 2 accounts

PASS 403 insufficient_scope – the API enforced scope

Try this: change `scope` in the call to `getCode()` to just `'accounts:read'`. The response has no `id_token` any more, because you stopped asking for `openid` — that one scope is the whole of OpenID Connect.

LAB 2

The PKCE transform, step by step

```
node client.js pkce
```

Prints a verifier, its SHA-256 in hex and base64, and the resulting challenge, so you can watch base64 become base64url. Then it verifies a correct verifier and a different one.

PASS correct verifier -> true

PASS a different one -> false (pkce_mismatch)

Try this: in `crypto-lab.js`, change `update(verifier, 'ascii')` to hash the raw random bytes instead of the encoded text. Lab 1 now fails with `invalid_grant` and nothing tells you why — the most common PKCE bug, reproduced in one edit.

LAB 3

A stolen authorization code, refused by PKCE

```
node client.js attack
```

Takes a legitimately issued code and hands it to an attacker, who tries to redeem it with no verifier and then with an invented one. Uses the **public** client, so there is no client secret quietly doing the work.

PASS REFUSED: `invalid_grant`

PASS REFUSED: the hash of a guess never matches

Why it matters: the leak still happened. PKCE does not prevent the code from escaping — it removes the value of having it.

Labs 4–6: breaking the rules

Each of these turns off a requirement of OAuth 2.1 and shows you what the requirement was for. Lab 4 is the one to run twice.

LAB 4

The same theft, with PKCE turned off

```
node client.js nopkce
```

Sets `require_pkce` to `false` on the running server, obtains a code with no `code_challenge` at all, and lets the attacker redeem it. It restores the safe default when it finishes.

FAIL ATTACK SUCCEEDED – the attacker holds a real access token

FAIL and it WORKS: `{"id":"ACC-4410","balance":4820.55}`

Run Lab 3 and Lab 4 back to back. Identical theft, opposite outcomes, one policy flag apart. Nothing else in this book makes the case for PKCE as quickly.

LAB 5

Replaying an already-used code

```
node client.js replay
```

Redeems one code twice. The second attempt is refused, and every token the first redemption produced is revoked — RFC 9700's answer to a code that may be in two places at once.

PASS tokens issued

PASS REFUSED: codes are single-use

Try this: `curl -X POST localhost:7000/policy -d 'key=single_use_codes&value=false'`, then run it again and watch the replay mint a second set of tokens.

LAB 6

Refresh rotation and reuse detection

```
node client.js refresh
```

Refreshes once and shows that a *different* refresh token comes back, then replays the retired one as a thief would. The whole family is revoked, including the copy the honest client was still holding.

PASS ROTATED – a brand-new refresh token came back

PASS REFUSED and the whole family revoked

PASS also refused – the server cannot tell thief from victim

Why it matters: the user gets logged out. That is not the bug, that is the feature — a silent permanent compromise became a loud recoverable one.

Labs 7–8: scope, and what a token is for

Two labs on the questions an API has to answer after the signature checks out.

LAB 7

Scope is capped three times over

```
node client.js scope
```

Signs in as **bob**, who is not entitled to `payments:write`, and asks for it anyway. The consent screen cannot offer what the user does not hold, so the token comes back narrower than requested — then the API refuses the operation.

```
requested openid profile accounts:read payments:write
actually granted openid profile accounts:read
PASS payments:write was dropped – the user never had it to give
PASS 403 insufficient_scope
```

Try this: run it again as `alice`, who *is* entitled. The payment now succeeds with `201`. Same client, same code, same request — different resource owner.

LAB 8

An ID token is not an access token

```
node client.js idtoken
```

Compares the two tokens one flow returns, then commits the classic mistake: sending the ID token to the API as a Bearer token. Finally it uses the `resource` parameter (RFC 8707) to mint a genuine, unexpired token for a *different* API and presents that instead.

```
access typ / aud at+jwt / http://localhost:7010
id typ / aud JWT / demo-web-app
PASS 401 unexpected_typ – refused, as it must be
PASS 401 audience_mismatch – refused
```

Why it matters: the second token has a perfect signature from the right issuer and has not expired. The only thing wrong with it is `aud` — which is precisely what confines a token to one API.

Labs 9–10: no user, and life after revocation

The last two labs cover the grant with no human in it, and the trade-off that explains every short token lifetime you have ever seen.

LAB 9

Machine to machine: no user, no consent

```
node client.js m2m
```

Gets a token with `client_credentials` — no browser, no redirect, no consent screen — then asks the API who the user is.

```
sub demo-service
refresh_token none (correct)
PASS 404 no_such_subject - correct, nobody delegated
```

Why it matters: `sub` is the client id. There is no human behind this token and nothing was delegated, which is why it must never be used to act on a user's behalf. No refresh token either: the client can simply ask again with its own credentials.

LAB 10

Revocation: local validation vs introspection

```
node client.js revoke
```

Revokes an access token, then calls two endpoints that serve the same data — one validating the JWT locally, one asking the authorization server.

```
local validation 200 <- STILL ACCEPTED
introspection 401 <- correctly refused
```

Why it matters: a self-contained JWT knows nothing about having been revoked; its signature and `exp` are unchanged. Introspection sees the truth at the cost of a round trip per call. **That** is why access tokens live for minutes: the lifetime is the compromise between the two.

Where to go from here

Every policy flag in `authz-server.js` can be switched at runtime — `curl localhost:7000/policy` lists them. Turn one off, re-run the labs, and read the warnings the server prints. Then try the exercises: add a `/userinfo` endpoint, enforce `max_purchase`-style limits, or implement DPoP (RFC 9449) so a stolen token is useless without the client's key.

Lab 11: OpenID Connect

The lab behind Chapter 8. One command walks the whole identity layer: the extra scope, the extra token, every check a client must run on it, and what disappears when you leave the scope out.

LAB 11

Who is this? (OpenID Connect)

```
node client.js oidc
```

Requests `openid profile email` with a `nonce`, then puts the access token and the ID token side by side so you can see they are addressed to different parties.

```
access typ / aud at+jwt / http://localhost:7010
id      typ / aud JWT / demo-web-app
```

Then it validates the ID token properly — signature by `kid`, `iss`, `aud`, `exp`, and the `nonce` it generated itself:

```
PASS signature, iss, aud and exp all check out
PASS nonce matches the one we generated – not a replayed token
sub u-1001 <- the stable identifier. Key your user on this.
```

And shows the two failures that matter:

```
PASS REFUSED: audience_mismatch – aud is checked, not assumed
PASS 403 insufficient_scope at /userinfo (no openid scope)
```

Exercises

1. **Break the nonce check.** In step 3, compare against a fresh random value instead of `f.sentNonce`. The lab now reports a mismatch — which is exactly what a replayed ID token would look like.
2. **Drop a scope.** Change the request to `openid` alone. The ID token loses `name` and `email`, and `/userinfo` returns `sub` and nothing else. Claims follow consent, not convenience.
3. **Send the ID token to the API** instead of the access token. It is refused on `typ` — the same mistake Lab 8 makes on purpose, seen from the identity side.
4. **Look at `auth_time` after a refresh.** Refresh the access token (Lab 6), and note that no new ID token arrives. Identity was asserted once, at login; a refresh grant does not re-assert it.

Labs 12–13: an agent, and its limits

The four labs behind Chapter 15. They use two more registered clients — `demo-agent`, which has no browser, and `demo-orchestrator`, the backend it calls.

LAB 12

An agent with no browser

```
node client.js agent-device
```

Runs the device grant end to end: the agent requests a user code, polls and is told to wait, polls too fast and is told to slow down, then the human approves at the IdP and the next poll returns tokens.

```
user_code PCRf-GFSB
PASS authorization_pending - correct, it must keep waiting
PASS slow_down - the interval is enforced
PASS tokens issued to the agent
sub u-1001 (the USER) client_id demo-agent (the agent)
```

Notice: the token acts for alice but records the agent in `client_id`. That pair is what makes agent activity auditable.

LAB 13

Limits a scope cannot express

```
node client.js agent-rar
```

The agent holds `payments:write` and an `authorization_details` grant capped at 100.00 from one account. Three payments are attempted against it.

```
PASS 201 - payment of 50 accepted
PASS 403 insufficient_authorization (5000 > 100.00)
PASS 403 insufficient_authorization (wrong source account)
```

Try this: delete the `authorization_details` argument in the lab and run it again. All three now succeed — the scope alone never had anything to say about the amount.

Labs 14–15: delegation chains

What happens when an agent is not the last hop — and what stops a trusted service from quietly acquiring authority nobody granted it.

LAB 14

Delegation chains and the act claim

```
node client.js agent-exchange
```

The agent hands its token to a backend service, which exchanges it (RFC 8693) for one scoped to the next hop. The API then reports the chain it can see.

```
sub u-1001 (still alice - the authority is hers)
act {"sub":"demo-orchestrator"} <- who is acting
acting_chain ["demo-orchestrator"]
```

Why it matters: without `act`, that request is indistinguishable from alice at a browser.

LAB 15

A confused deputy tries to escalate

```
node client.js agent-deputy
```

The agent is delegated `accounts:read` only. The backend then tries to exchange that token for permission to move money.

```
PASS REFUSED: invalid_scope
    Cannot widen scope on exchange: payments:write
PASS 403 insufficient_scope at the API too
```

Notice the defence in depth: the authorization server refused to widen the grant, and the API independently refused a scope the token never carried. Either check alone would have held.

PKCE and JWT primitives

Everything cryptographic the labs need, on Node core modules. Read `verifyPkce` and `verifyJwt` and you have read the two checks the rest of the protocol rests on.

LINES 1–42 OF 126

```

1  'use strict';
2  // =====
3  // LAB FILE 1 of 4 - crypto-lab.js
4  // PKCE and JWT primitives, on Node core modules only. No dependencies.
5  // =====
6  const crypto = require('node:crypto');
7
8  // --- base64url: base64 with +/ -> _- and the = padding removed -----
9  const b64u = (buf) => Buffer.from(buf).toString('base64url');
10 const unb64u = (str) => Buffer.from(str, 'base64url');
11
12 // --- PKCE (RFC 7636) -----
13
14 // The verifier is the secret: 32 random bytes, base64url -> 43 characters.
15 const createVerifier = () => crypto.randomBytes(32).toString('base64url');
16
17 // challenge = BASE64URL(SHA256(ASCII(verifier))) <- hash the TEXT, not bytes
18 const challengeS256 = (verifier) =>
19   crypto.createHash('sha256').update(verifier, 'ascii').digest('base64url');
20
21 // Show every step of the transform, for Lab 2.
22 function explainPkce(verifier) {
23   const digest = crypto.createHash('sha256').update(verifier, 'ascii').digest();
24   return {
25     code_verifier: verifier,
26     length: verifier.length,
27     sha256_hex: digest.toString('hex'),
28     sha256_base64: digest.toString('base64'), // note the trailing '='
29     code_challenge: digest.toString('base64url'), // the same, url-safe
30   };
31 }
32
33 // The authorization server's check. Constant-time: a comparison that returns
34 // early on the first differing byte leaks the stored challenge.
35 function verifyPkce(verifier, challenge, method) {
36   if (!verifier) return { ok: false, reason: 'missing_code_verifier' };
37   if (!/^[A-Za-z0-9\-\._~]{43,128}$/.test(verifier))
38     return { ok: false, reason: 'malformed_code_verifier' };
39   const computed = method === 'plain' ? verifier : challengeS256(verifier);
40   const a = Buffer.from(computed), b = Buffer.from(challenge || '');
41   const ok = a.length === b.length && crypto.timingSafeEqual(a, b);
42   return ok ? { ok: true, computed } : { ok: false, reason: 'pkce_mismatch', computed };

```

PKCE and JWT primitives

LINES 43–84 OF 126

```

43 }
44
45 // --- Keys and JWTs -----
46
47 // The key id is a SHA-256 over the key's own material (RFC 7638 thumbprint),
48 // so new key material always announces itself with a new kid. That is what
49 // lets the API cache public keys safely. Restarting this server rotates keys.
50 function generateKey() {
51   const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa', {
52     modulusLength: 2048,
53     publicKeyEncoding: { type: 'spki', format: 'pem' },
54     privateKeyEncoding: { type: 'pkcs8', format: 'pem' },
55   });
56   const jwk = crypto.createPublicKey(publicKey).export({ format: 'jwk' });
57   const thumb = crypto.createHash('sha256')
58     .update(JSON.stringify({ e: jwk.e, kty: jwk.kty, n: jwk.n }))
59     .digest('base64url');
60   const kid = 'lab-' + thumb.slice(0, 16);
61   return { kid, privateKey, publicJwk: { ...jwk, alg: 'RS256', use: 'sig', kid } };
62 }
63
64 // A JWT is three base64url segments joined by dots: header.payload.signature.
65 function signJwt(payload, key, typ) {
66   const header = { alg: 'RS256', typ: typ || 'JWT', kid: key.kid };
67   const input = b64u(JSON.stringify(header)) + '.' + b64u(JSON.stringify(payload));
68   const sig = crypto.sign('RSA-SHA256', Buffer.from(input), key.privateKey);
69   return input + '.' + sig.toString('base64url');
70 }
71
72 // Decode WITHOUT verifying. For display only – never trust this output.
73 function decodeJwt(token) {
74   const p = String(token || '').split('.');
75   if (p.length !== 3) return null;
76   try {
77     return {
78       header: JSON.parse(unb64u(p[0]).toString('utf8')),
79       payload: JSON.parse(unb64u(p[1]).toString('utf8')),
80       signingInput: p[0] + '.' + p[1],
81       signature: p[2],
82     };
83   } catch { return null; }
84 }

```

PKCE and JWT primitives

LINES 85–126 OF 126

```

85
86 // Verify the way a resource server must: signature first, then every claim
87 // that scopes the token to THIS api at THIS moment. See Chapter 11.
88 function verifyJwt(token, { getKey, issuer, audience, requiredTyp, skew = 5 }) {
89   const d = decodeJwt(token);
90   if (!d) return { valid: false, reason: 'malformed_token' };
91   // Never let the token choose its own algorithm: that is how alg:none and
92   // RS256→HS256 confusion attacks start.
93   if (d.header.alg !== 'RS256') return { valid: false, reason: 'unsupported_alg' };
94   if (requiredTyp && d.header.typ !== requiredTyp)
95     return { valid: false, reason: 'unexpected_typ' };
96   const key = getKey(d.header.kid);
97   if (!key) return { valid: false, reason: 'unknown_kid' };
98   const ok = crypto.verify('RSA-SHA256', Buffer.from(d.signingInput),
99                           key, unb64u(d.signature));
100   if (!ok) return { valid: false, reason: 'bad_signature' };
101
102   const now = Math.floor(Date.now() / 1000);
103   const p = d.payload;
104   if (issuer && p.iss !== issuer) return { valid: false, reason: 'issuer_mismatch' };
105   if (audience && ![].concat(p.aud || []).includes(audience))
106     return { valid: false, reason: 'audience_mismatch' };
107   if (typeof p.exp === 'number' && now > p.exp + skew)
108     return { valid: false, reason: 'token_expired' };
109   return { valid: true, payload: p, header: d.header };
110 }
111
112 // --- odds and ends -----
113 const randomToken = (n = 32) => crypto.randomBytes(n).toString('base64url');
114 const randomId = (n = 8) => crypto.randomBytes(n).toString('hex');
115 const sha256hex = (v) => crypto.createHash('sha256').update(v).digest('hex');
116
117 function safeEqual(a, b) {
118   const x = Buffer.from(String(a ?? '')), y = Buffer.from(String(b ?? ''));
119   return x.length === y.length && crypto.timingSafeEqual(x, y);
120 }
121
122 module.exports = {
123   b64u, unb64u, createVerifier, challengeS256, explainPkce, verifyPkce,
124   generateKey, signJwt, decodeJwt, verifyJwt,
125   randomToken, randomId, sha256hex, safeEqual,
126 };

```

The authorization server and IdP

The whole of Chapter 4 as running code: `/authorize` with its six validations, login, consent, and a `/token` endpoint handling three grants. The `policy` object near the top is what Lab 4 switches off.

LINES 1–53 OF 850

```

1  'use strict';
2  // =====
3  // LAB FILE 2 of 4 – authz-server.js      run:  node authz-server.js
4  // A complete OAuth 2.1 authorization server + identity provider.
5  // Implements: RFC 6749 (as narrowed by 2.1), 7636 PKCE, 8414 discovery,
6  //             7662 introspection, 7009 revocation, 9068 at+jwt, 9700 BCP.
7  // =====
8  const http = require('node:http');
9  const C = require('./crypto-lab');
10
11  const PORT = 7000;
12  const ISSUER = 'http://localhost:7000'; // browser-facing identity; goes in `iss`
13  const API = 'http://localhost:7010';   // the audience we mint tokens for
14
15  // Grant types are URNs, not short names, once you leave the core spec.
16  const DEVICE_GRANT = 'urn:ietf:params:oauth:grant-type:device_code';
17  const EXCHANGE_GRANT = 'urn:ietf:params:oauth:grant-type:token-exchange';
18  const ACCESS_TOKEN_TYPE = 'urn:ietf:params:oauth:token-type:access_token';
19
20  // --- who can log in -----
21  const USERS = {
22    alice: { sub: 'u-1001', password: 'wonderland', name: 'Alice Liddell',
23             email: 'alice@lab.test',
24             can: ['openid', 'profile', 'email', 'accounts:read', 'payments:write'] },
25    // bob deliberately cannot delegate payments:write – see Lab 6.
26    bob: { sub: 'u-1002', password: 'builder', name: 'Bob Builder',
27           email: 'bob@lab.test',
28           can: ['openid', 'profile', 'email', 'accounts:read'] },
29  };
30  const userBySub = (sub) => Object.values(USERS).find((u) => u.sub === sub);
31  const usernameOf = (sub) =>
32    Object.keys(USERS).find((k) => USERS[k].sub === sub) || null;
33
34  // --- which apps are registered -----
35  const CLIENTS = {
36    // A server-side app CAN keep a secret, so it authenticates to /token.
37    'demo-web-app': { secret: 'web-app-secret', type: 'confidential',
38                     redirect_uris: ['http://localhost:7020/callback'],
39                     grants: ['authorization_code', 'refresh_token'],
40                     scopes: ['openid', 'profile', 'email', 'accounts:read', 'payments:write'] },
41    // A browser or mobile app CANNOT. PKCE is its only proof of identity.
42    'demo-spa': { secret: null, type: 'public',
43                 redirect_uris: ['http://localhost:7020/spa-callback'],
44                 grants: ['authorization_code', 'refresh_token'],
45                 scopes: ['openid', 'profile', 'accounts:read'] },
46    // No user involved at all: the app acts as itself.
47    'demo-service': { secret: 'service-secret', type: 'confidential',
48                     redirect_uris: [], grants: ['client_credentials'],
49                     scopes: ['accounts:read', 'profile'] },
50
51    // --- agentic access (Chapter 15) -----
52    // An autonomous agent has no browser, so it cannot use a redirect. It asks
53    // for a code the human types in elsewhere: the device grant, RFC 8628.

```

The authorization server and IdP

LINES 54–110 OF 850

```

54 // Public, because an agent binary or container cannot keep a secret either.
55 'demo-agent': { secret: null, type: 'public', redirect_uris: [],
56   grants: [DEVICE_GRANT, 'refresh_token'],
57   scopes: ['openid', 'profile', 'accounts:read', 'payments:write'] },
58
59 // The service an agent calls, which must then call the API *as the user*.
60 // It swaps the user's token for a downstream one (RFC 8693) that records
61 // who is acting, so the API can see the whole chain.
62 'demo-orchestrator': { secret: 'orchestrator-secret', type: 'confidential',
63   redirect_uris: [], grants: [EXCHANGE_GRANT, 'client_credentials'],
64   scopes: ['accounts:read', 'payments:write'] },
65 };
66
67 const SCOPES = ['openid', 'profile', 'email', 'accounts:read', 'payments:write'];
68
69 // --- the safety rules. Every one is ON in a correct OAuth 2.1 server. -----
70 // Lab 3 turns require_pkce off and watches a stolen code start working.
71 const policy = {
72   require_pkce: true,           // PKCE for every authorization_code request
73   exact_redirect_uri: true,     // string equality, no wildcards, no prefixes
74   single_use_codes: true,       // a replayed code is revoked, not honoured
75   rotate_refresh: true,         // a new refresh token on every refresh
76   detect_reuse: true,           // a replayed refresh token kills the whole family
77 };
78
79 const TTL = { code: 60, access: 300, refresh: 3600, session: 1800, device: 300 };
80
81 // --- server-side state (a real AS keeps this in a database) -----
82 const sessions = new Map();    // browser is logged in to the IdP
83 const requests = new Map();    // an /authorize call parked during login
84 const codes = new Map();       // one-time authorization codes
85 const refresh = new Map();     // sha256(token) -> record, grouped into families
86 const revoked = new Set();     // access-token ids killed before exp
87 const devices = new Map();     // device_code -> a pending agent authorization
88
89 const KEY = C.generateKey();
90 const now = () => Math.floor(Date.now() / 1000);
91 const splitScope = (s) => String(s || '').trim().split(/\s+/).filter(Boolean);
92
93 // --- tiny http helpers -----
94 function readBody(req) {
95   return new Promise((resolve) => {
96     let raw = '';
97     req.on('data', (c) => { raw += c; if (raw.length > 1e5) req.destroy(); });
98     req.on('end', () => resolve(new URLSearchParams(raw)));
99   });
100 }
101 const cookies = (req) => Object.fromEntries((req.headers.cookie || '').
102   .split(';').map((p) => p.trim().split('=')).filter((p) => p[0]));
103
104 function json(res, status, obj) {
105   res.writeHead(status, { 'content-type': 'application/json',
106     'cache-control': 'no-store' }); // these are credentials
107   res.end(JSON.stringify(obj, null, 2));
108 }
109 const html = (res, body, status = 200) =>
110   (res.writeHead(status, { 'content-type': 'text/html; charset=utf-8' })),

```

The authorization server and IdP

LINES 111–167 OF 850

```

111     res.end('<!doctype html><meta charset=utf-8>' +
112           '<body style="font:15px/1.5 system-ui;max-width:34em;margin:3em auto">' +
113           body));
114 const redirect = (res, location) => (res.writeHead(302, { location }), res.end());
115 const esc = (s) => String(s).replace(/[\<>"]/g, (c) =>
116   ({ '<': '&lt;'; '>': '&gt;'; '&': '&amp;'; '"': '&quot;'; }[c]));
117
118 // Deliver an error to the client by redirect. Only ever called AFTER the
119 // client_id and redirect_uri are both validated – otherwise this endpoint
120 // would be an open redirector.
121 function redirectError(res, uri, error, description, state) {
122   const u = new URL(uri);
123   u.searchParams.set('error', error);
124   if (description) u.searchParams.set('error_description', description);
125   if (state) u.searchParams.set('state', state);
126   log('reject', error + ': ' + description);
127   return redirect(res, u.toString());
128 }
129 const log = (tag, msg) => console.log('  [as] ' + tag.padEnd(7) + msg);
130
131 // --- client authentication -----
132 function authenticateClient(req, body) {
133   const auth = req.headers.authorization || '';
134   let id, secret;
135   if (auth.toLowerCase().startsWith('basic ')) {
136     const raw = Buffer.from(auth.slice(6), 'base64').toString('utf8');
137     const i = raw.indexOf(':');
138     id = decodeURIComponent(raw.slice(0, i));
139     secret = decodeURIComponent(raw.slice(i + 1));
140   } else { id = body.get('client_id'); secret = body.get('client_secret'); }
141
142   const client = CLIENTS[id];
143   if (!client) return { error: 'invalid_client', description: 'Unknown client_id.' };
144   if (client.type === 'confidential') {
145     if (!C.safeEqual(secret, client.secret))
146       return { error: 'invalid_client', description: 'Client authentication failed.' };
147     log('ok', 'client authenticated: ' + id);
148   } else {
149     // A public client is IDENTIFIED, never authenticated. PKCE carries the proof.
150     log('info', 'public client identified: ' + id + ' (PKCE carries the proof)');
151   }
152   return { id, client };
153 }
154
155 // --- token minting -----
156 function issueAccessToken(clientId, sub, scope, familyId, audience, extra) {
157   const iat = now(), jti = C.randomId(12);
158   const claims = {
159     iss: ISSUER, sub, aud: audience || API, client_id: clientId,
160     scope: scope.join(' '), jti, iat, nbf: iat, exp: iat + TTL.access,
161   };
162   // `act` records WHO is acting on the subject's behalf (RFC 8693 4.1). The
163   // API can then log and authorize the whole chain, not just the end user.
164   if (extra && extra.act) claims.act = extra.act;
165   // `authorization_details` carries structured, per-request limits that a
166   // coarse scope string cannot express (RFC 9396). See Chapter 15.
167   if (extra && extra.details) claims.authorization_details = extra.details;

```

The authorization server and IdP

LINES 168–224 OF 850

```

168     const token = C.signJwt(claims, KEY, 'at+jwt');
169     return { token, jti, familyId };
170 }
171
172 // Parse and sanity-check an authorization_details value (RFC 9396).
173 function parseDetails(raw) {
174     if (!raw) return null;
175     let d;
176     try { d = JSON.parse(raw); } catch { return { error: 'not JSON' }; }
177     if (!Array.isArray(d)) return { error: 'must be a JSON array' };
178     for (const entry of d) {
179         if (!entry || typeof entry !== 'object' || !entry.type)
180             return { error: 'each entry needs a "type"' };
181     }
182     return { value: d };
183 }
184
185 // A short code a human can read off a screen and type somewhere else.
186 // The alphabet omits characters people confuse: 0/O, 1/I, etc.
187 function makeUserCode() {
188     const AB = 'BCDFGHJKLMNPQRSTVWXZ';
189     let out = '';
190     for (let i = 0; i < 8; i++) out += AB[Math.floor(Math.random() * AB.length)];
191     return out.slice(0, 4) + '-' + out.slice(4);
192 }
193
194 function issueRefreshToken(clientId, sub, scope, familyId, generation) {
195     const token = C.randomToken(32);
196     // Stored hashed, like a password: a database leak then yields nothing usable.
197     refresh.set(C.sha256hex(token), { clientId, sub, scope, familyId, generation,
198         used: false, revoked: false, expiresAt: now() + TTL.refresh });
199     return token;
200 }
201
202 function revokeFamily(familyId) {
203     let n = 0;
204     for (const r of refresh.values())
205         if (r.familyId === familyId && !r.revoked) { r.revoked = true; n++; }
206     return n;
207 }
208
209 // =====
210 // The server
211 // =====
212 const server = http.createServer(async (req, res) => {
213     const url = new URL(req.url, ISSUER);
214     const q = url.searchParams;
215     const path = url.pathname;
216     const body = req.method === 'POST' ? await readBody(req) : new URLSearchParams();
217
218     // ----- discovery + keys (RFC 8414) -----
219     if (path === '/.well-known/oauth-authorization-server') return json(res, 200, {
220         issuer: ISSUER,
221         authorization_endpoint: ISSUER + '/authorize',
222         token_endpoint: ISSUER + '/token',
223         // RFC 8628 requires advertising this, or an agent cannot discover it.
224         device_authorization_endpoint: ISSUER + '/device_authorization',

```

The authorization server and IdP

LINES 225–281 OF 850

```

225 // The OpenID Connect half of this server (Chapter 8).
226 userinfo_endpoint: ISSUER + '/userinfo',
227 subject_types_supported: ['public'],
228 id_token_signing_alg_values_supported: ['RS256'],
229 claims_supported: ['sub', 'name', 'preferred_username', 'email',
230   'email_verified', 'auth_time', 'nonce'],
231 jwks_uri: ISSUER + '/jwks.json',
232 introspection_endpoint: ISSUER + '/introspect',
233 revocation_endpoint: ISSUER + '/revoke',
234 scopes_supported: SCOPES,
235 // OAuth 2.1: `code` only. The implicit grant is gone, so no response type
236 // can return a token in the front channel.
237 response_types_supported: ['code'],
238 grant_types_supported: ['authorization_code', 'refresh_token',
239   'client_credentials', DEVICE_GRANT, EXCHANGE_GRANT],
240 // Tell clients we understand structured grants (RFC 9396), and which
241 // kinds. An agent can then ask for limits instead of broad scopes.
242 authorization_details_types_supported: ['payment_initiation'],
243 code_challenge_methods_supported: ['S256'], // never offer `plain`
244 token_endpoint_auth_methods_supported: ['client_secret_basic', 'none'],
245 });
246
247 if (path === '/jwks.json') return json(res, 200, { keys: [KEY.publicJwk] });
248 if (path === '/healthz')
249   return json(res, 200, { ok: true, issuer: ISSUER, kid: KEY.kid });
250
251 // ----- the lab switchboard -----
252 if (path === '/policy' && req.method === 'GET') return json(res, 200, policy);
253 if (path === '/policy' && req.method === 'POST') {
254   const key = body.get('key'), value = body.get('value') === 'true';
255   if (!(key in policy)) return json(res, 400, { error: 'unknown_policy_key' });
256   policy[key] = value;
257   log('policy', key + ' = ' + value + (value ? '' : ' << now running UNSAFELY'));
258   return json(res, 200, policy);
259 }
260
261 // ----- /authorize : the FRONT CHANNEL -----
262 // Runs in the browser. Everything here is visible in the URL bar, history,
263 // logs and Referer headers, so nothing secret may travel on it.
264 if (path === '/authorize') {
265   const client = CLIENTS[q.get('client_id')];
266   // Step 1: identify the client. Not yet trusted – just recognised.
267   if (!client) return html(res,
268     '<h2>invalid_client</h2><p>Unknown client_id. The authorization server will '
269     + 'not redirect an error for an unknown client, because it has no registered '
270     + 'URI it can trust.</p>', 400);
271
272   // Step 2: validate redirect_uri BEFORE trusting it.
273   const uri = q.get('redirect_uri') || '';
274   const match = policy.exact_redirect_uri
275     ? client.redirect_uris.includes(uri)
276     : client.redirect_uris.some((r) => uri.startsWith(r.split('/callback')[0]));
277   if (!match) return html(res,
278     '<h2>invalid_request</h2><p>redirect_uri <code>' + esc(uri) + '</code> is not '
279     + 'registered. This check is what stops an attacker having your authorization '
280     + 'code delivered to their own server.</p>', 400);
281   if (!policy.exact_redirect_uri)

```

The authorization server and IdP

LINES 282–338 OF 850

```

282     log('warn', 'redirect_uri matched by PREFIX (unsafe)');
283
284     const state = q.get('state');
285
286     // Step 3: response_type. OAuth 2.1 means `code`, full stop.
287     if (q.get('response_type') !== 'code') return redirectError(res, uri,
288       'unsupported_response_type',
289       'OAuth 2.1 supports only response_type=code. The implicit grant was removed '
290       + 'because it delivered access tokens through the browser URL.', state);
291
292     // Step 4: PKCE. The defining requirement of OAuth 2.1.
293     const challenge = q.get('code_challenge');
294     const method = q.get('code_challenge_method');
295     if (!challenge) {
296       if (policy.require_pkce) return redirectError(res, uri, 'invalid_request',
297         'code_challenge is required. OAuth 2.1 mandates PKCE for every '
298         + 'authorization code request, for public and confidential clients alike.',
299         state);
300       log('warn', 'PROCEEDING WITHOUT PKCE - this code will be stealable');
301     } else {
302       if (!method) return redirectError(res, uri, 'invalid_request',
303         'Send code_challenge_method=S256 explicitly. If omitted, RFC 7636 '
304         + 'defaults to "plain", which offers no protection.', state);
305       if (method !== 'S256') return redirectError(res, uri, 'invalid_request',
306         'code_challenge_method must be S256. With "plain" the challenge IS the '
307         + 'verifier, so anyone who sees the request can redeem the code.', state);
308       if (!/^[A-Za-z0-9\-\_]{43}$/.test(challenge)) return redirectError(res, uri,
309         'invalid_request', 'An S256 code_challenge is 43 base64url characters. '
310         + 'Did you send the verifier by mistake?', state);
311       log('ok', 'PKCE challenge stored: ' + challenge.slice(0, 16) + '...');
312     }
313
314     // Step 5: scope, capped by what this client is registered for.
315     const wanted = splitScope(q.get('scope'));
316     if (!wanted.length) return redirectError(res, uri, 'invalid_scope',
317       'No scope requested.', state);
318     const bad = wanted.filter((s) => !client.scopes.includes(s));
319     if (bad.length) return redirectError(res, uri, 'invalid_scope',
320       'Client is not registered for: ' + bad.join(' '), state);
321
322     // Step 6: park the request and hand over to the IdP.
323     const rid = C.randomId(16);
324     requests.set(rid, { clientId: q.get('client_id'), uri, scope: wanted, state,
325       nonce: q.get('nonce'), challenge, method });
326     return resume(res, rid, cookies(req).lab_session);
327   }
328
329   // ----- IdP: authentication -----
330   if (path === '/login' && req.method === 'POST') {
331     const rid = body.get('request_id'), r = requests.get(rid);
332     if (!r) return html(res, '<h2>Expired</h2><p>Start again.</p>', 400);
333     const user = USERS[String(body.get('username') || '').toLowerCase()];
334     // One generic message for both cases: telling them apart is a free
335     // user-enumeration oracle for an attacker.
336     if (!user || !C.safeEqual(body.get('password'), user.password)) {
337       log('reject', 'authentication failed');
338       return loginPage(res, rid, r, 'That username and password is not correct.');
```

The authorization server and IdP

LINES 339–395 OF 850

```

339     }
340     const sid = C.randomToken(24);
341     sessions.set(sid, { sub: user.sub, authTime: now(),
342                       expiresAt: now() + TTL.session });
343     log('ok', 'user authenticated: ' + user.name);
344     res.setHeader('set-cookie', 'lab_session=' + sid
345       + '; Path=/; HttpOnly; SameSite=Lax; Max-Age=' + TTL.session);
346     return resume(res, rid, sid);
347   }
348
349   // ----- OAuth: consent, then the code -----
350   if (path === '/consent' && req.method === 'POST') {
351     const rid = body.get('request_id'), r = requests.get(rid);
352     if (!r) return html(res, '<h2>Expired</h2>', 400);
353     const s = sessions.get(cookies(req).lab_session);
354     if (!s) return resume(res, rid, null);
355     const user = userBySub(s.sub);
356
357     if (body.get('action') !== 'allow') {
358       requests.delete(rid);
359       return redirectError(res, r.uri, 'access_denied',
360         'The resource owner refused. Every client must handle this.', r.state);
361     }
362     // Approved = asked for AND the user holds it AND the user ticked it.
363     const ticked = body.getAll('scope');
364     const granted = r.scope.filter(x =>
365       user.can.includes(x) && (ticked.includes(x) || x === 'openid'));
366     if (!granted.length) {
367       requests.delete(rid);
368       return redirectError(res, r.uri, 'access_denied', 'No scopes granted.', r.state);
369     }
370     if (granted.length !== r.scope.length)
371       log('warn', 'granted scope is NARROWER than requested: ' + granted.join(' '));
372
373     // Bind to the code everything /token must re-check: the client, the
374     // redirect_uri, and the PKCE challenge. The code itself means nothing.
375     const code = C.randomToken(32);
376     codes.set(code, { clientId: r.clientId, sub: s.sub, scope: granted, uri: r.uri,
377       challenge: r.challenge, method: r.method, nonce: r.nonce,
378       authTime: s.authTime, used: false, expiresAt: now() + TTL.code });
379     requests.delete(rid);
380     log('ok', 'code issued: ' + code.slice(0, 12) + '... scope=' + granted.join(' '));
381
382     const out = new URL(r.uri);
383     out.searchParams.set('code', code);
384     if (r.state) out.searchParams.set('state', r.state);
385     out.searchParams.set('iss', ISSUER); // RFC 9207: mix-up defence
386     return redirect(res, out.toString());
387   }
388
389   // ----- agentic access: the device grant (RFC 8628) -----
390   // An agent with no browser asks here first. It gets a code the HUMAN types
391   // in somewhere else, on a device that does have a browser.
392   if (path === '/device_authorization' && req.method === 'POST') {
393     const auth = authenticateClient(req, body);
394     if (auth.error) return json(res, 401, { error: auth.error });
395     if (!auth.client.grants.includes(DEVICE_GRANT))

```

The authorization server and IdP

LINES 396–452 OF 850

```

396     return json(res, 400, { error: 'unauthorized_client' });
397
398     const asked = splitScope(body.get('scope'));
399     const bad = asked.filter((x) => !auth.client.scopes.includes(x));
400     if (bad.length) return json(res, 400, { error: 'invalid_scope',
401       error_description: 'Not registered for: ' + bad.join(' ') });
402
403     // An agent may also ask for structured limits, not just scopes.
404     const parsed = parseDetails(body.get('authorization_details'));
405     if (parsed && parsed.error) return json(res, 400, {
406       error: 'invalid_authorization_details', error_description: parsed.error });
407
408     const deviceCode = C.randomToken(32), userCode = makeUserCode();
409     devices.set(deviceCode, { userCode, clientId: auth.id, scope: asked,
410       details: parsed ? parsed.value : null, status: 'pending', sub: null,
411       lastPoll: 0, expiresAt: now() + TTL.device });
412     log('ok', 'device authorization started, user_code=' + userCode);
413     return json(res, 200, {
414       device_code: deviceCode,
415       user_code: userCode,
416       verification_uri: ISSUER + '/device',
417       verification_uri_complete: ISSUER + '/device?user_code=' + userCode,
418       expires_in: TTL.device,
419       interval: 5,           // the agent must not poll faster than this
420     });
421   }
422
423   // The page the human opens. In a production server this would reuse the
424   // normal session and consent screens; it is collapsed into one form here
425   // so the lab stays readable.
426   if (path === '/device' && req.method === 'GET') {
427     const pre = q.get('user_code') || '';
428     return html(res,
429       '<h2>Authorize a device</h2><p>An application is asking to act on your '
430       + 'behalf. Check the code shown on that device matches the one below, then '
431       + 'sign in to approve it.</p>'
432       + '<form method=POST action=/device>'
433       + '<p><label>Code<br><input name=user_code value="' + esc(pre) + '"></label>'
434       + '<p><label>Username<br><input name=username autofocus></label>'
435       + '<p><label>Password<br><input name=password type=password></label>'
436       + '<p><button name=action value=allow>Approve</button>'
437       + '<button name=action value=deny>Deny</button></form>'
438       + '<hr><p style="color:#666;font-size:13px">alice / wonderland</p>');
439   }
440
441   if (path === '/device' && req.method === 'POST') {
442     const code = String(body.get('user_code') || '').trim().toUpperCase();
443     let entry = null;
444     for (const d of devices.values()) if (d.userCode === code) entry = d;
445     if (!entry || entry.expiresAt < now())
446       return html(res, '<h2>Unknown or expired code</h2>', 400);
447
448     if (body.get('action') === 'deny') {
449       entry.status = 'denied';
450       log('warn', 'the human DENIED the device request');
451       return html(res, '<h2>Denied</h2><p>You can close this page.</p>');
452     }

```

The authorization server and IdP

LINES 453–509 OF 850

```

453     const user = USERS[String(body.get('username') || '').toLowerCase()];
454     if (!user || !C.safeEqual(body.get('password'), user.password))
455         return html(res, '<h2>Wrong username or password</h2>', 401);
456
457     // The user may hold less than the agent asked for. Narrow it here.
458     entry.scope = entry.scope.filter((x) => user.can.includes(x));
459     entry.sub = user.sub;
460     entry.status = 'approved';
461     log('ok', 'device approved by ' + user.name + ' scope=' + entry.scope.join(' '));
462     return html(res, '<h2>Approved</h2><p>The application can continue. You can '
463         + 'close this page.</p>');
464 }
465
466 // ----- /token : the BACK CHANNEL -----
467 if (path === '/token' && req.method === 'POST') {
468     const grant = body.get('grant_type');
469     log('step', 'POST /token grant_type=' + grant);
470
471     if (grant === 'password' || grant === 'implicit') return json(res, 400, {
472         error: 'unsupported_grant_type',
473         error_description: 'Removed in OAuth 2.1. The password grant made the app '
474             + 'handle the real password: no consent, no MFA, no federation. Use '
475             + 'authorization_code + PKCE.' });
476
477     const auth = authenticateClient(req, body);
478     if (auth.error) {
479         res.setHeader('www-authenticate', 'Basic realm="lab"');
480         return json(res, 401,
481             { error: auth.error, error_description: auth.description });
482     }
483     const { id: clientId, client } = auth;
484     if (!client.grants.includes(grant)) return json(res, 400, {
485         error: 'unauthorized_client',
486         error_description: 'Client is not registered for grant_type=' + grant });
487
488     // ===== authorization_code =====
489     if (grant === 'authorization_code') {
490         const code = body.get('code');
491         const entry = codes.get(code);
492         if (!entry) return json(res, 400, { error: 'invalid_grant',
493             error_description: 'Unknown authorization code.' });
494
495         // Replay detection. A code is a one-shot credential (RFC 9700 4.1.3).
496         if (entry.used) {
497             if (policy.single_use_codes) {
498                 const killed = entry.familyId ? revokeFamily(entry.familyId) : 0;
499                 log('reject', 'CODE REPLAY - revoked ' + killed + ' token(s) it produced');
500                 return json(res, 400, { error: 'invalid_grant',
501                     error_description: 'This code was already used. A replay means someone '
502                         + 'else may hold it, so every token derived from it is now revoked.' });
503             }
504             log('warn', 'CODE REPLAY ALLOWED - single_use_codes is off');
505         }
506         if (entry.expiresAt < now()) return json(res, 400, { error: 'invalid_grant',
507             error_description: 'Code expired. Codes live ~' + TTL.code + 's because they '
508                 + 'cross the browser.' });
509         if (entry.clientId !== clientId) {

```

The authorization server and IdP

LINES 510–566 OF 850

```

510     log('reject', 'code substitution blocked: issued to ' + entry.clientId);
511     return json(res, 400, { error: 'invalid_grant',
512       error_description: 'This code was issued to a different client.' });
513   }
514   if (entry.uri !== body.get('redirect_uri')) return json(res, 400, {
515     error: 'invalid_grant',
516     error_description: 'redirect_uri does not match the authorization request.' });
517
518   // ===== the PKCE check: prove you started this flow =====
519   if (entry.challenge) {
520     const r = C.verifyPkce(body.get('code_verifier'),
521       entry.challenge, entry.method);
522     if (!r.ok) {
523       entry.used = true; // burn it: something is wrong
524       log('reject', 'PKCE FAILED (' + r.reason + ') expected='
525         + entry.challenge.slice(0, 16) + '... got='
526         + String(r.computed).slice(0, 16));
527       return json(res, 400, { error: 'invalid_grant',
528         error_description: 'PKCE check failed (' + r.reason + '). The hash of the '
529         + 'presented code_verifier does not equal the stored code_challenge, so '
530         + 'whoever sent this did not start the flow.' });
531     }
532     log('ok', 'PKCE verified: SHA256(verifier) == stored challenge');
533   } else {
534     log('warn', 'no PKCE binding - anyone holding this code could redeem it');
535   }
536
537   entry.used = true;
538   const familyId = C.randomId(8);
539   entry.familyId = familyId;
540   const user = userBySub(entry.sub);
541   const at = issueAccessToken(clientId, entry.sub, entry.scope, familyId);
542   const out = { access_token: at.token, token_type: 'Bearer',
543     expires_in: TTL.access, scope: entry.scope.join(' ') };
544
545   if (client.grants.includes('refresh_token'))
546     out.refresh_token = issueRefreshToken(clientId, entry.sub, entry.scope,
547       familyId, 1);
548   if (entry.scope.includes('openid')) {
549     // An ID token is for the CLIENT: aud is the client_id, not the API.
550     const iat = now();
551     out.id_token = C.signJwt({ iss: ISSUER, sub: user.sub, aud: clientId,
552       iat, exp: iat + TTL.access, auth_time: entry.authTime,
553       nonce: entry.nonce || undefined,
554       name: entry.scope.includes('profile') ? user.name : undefined,
555       email: entry.scope.includes('email') ? user.email : undefined }, KEY, 'JWT');
556   }
557   log('ok', 'code exchanged for tokens');
558   return json(res, 200, out);
559 }
560
561 // ===== refresh_token =====
562 if (grant === 'refresh_token') {
563   const presented = body.get('refresh_token');
564   const hash = C.sha256hex(String(presented || ''));
565   const rec = refresh.get(hash);
566   if (!rec) return json(res, 400, { error: 'invalid_grant',

```

The authorization server and IdP

LINES 567–623 OF 850

```

567     error_description: 'Unknown refresh token.' });
568     if (rec.clientId !== clientId) return json(res, 400, { error: 'invalid_grant',
569       error_description: 'This refresh token belongs to a different client.' });
570     if (rec.revoked) return json(res, 400, { error: 'invalid_grant',
571       error_description: 'Revoked, most likely because a replay was detected '
572       + 'earlier in its family.' });
573
574     // ===== reuse detection: the whole point of rotation (RFC 9700 4.14) =====
575     if (rec.used) {
576       if (policy.detect_reuse) {
577         const killed = revokeFamily(rec.familyId);
578         log('reject', 'REFRESH REUSE - revoked family of ' + killed);
579         return json(res, 400, { error: 'invalid_grant',
580           error_description: 'This refresh token was already used (generation '
581             + rec.generation + '). Either it was stolen and replayed, or the real '
582             + 'client lost the rotated value. The server cannot tell, so the whole '
583             + 'family is revoked and the user must sign in again.' });
584       }
585       log('warn', 'reused refresh token ACCEPTED - detect_reuse is off');
586     }
587     if (rec.expiresAt < now()) return json(res, 400, { error: 'invalid_grant',
588       error_description: 'Refresh token expired.' });
589
590     // Scope may only ever shrink on refresh.
591     let scope = rec.scope;
592     if (body.get('scope')) {
593       const asked = splitScope(body.get('scope'));
594       const up = asked.filter((s) => !rec.scope.includes(s));
595       if (up.length) return json(res, 400, { error: 'invalid_scope',
596         error_description: 'Cannot widen scope on refresh: ' + up.join(' ') });
597       scope = asked;
598     }
599
600     rec.used = true;
601     const at = issueAccessToken(clientId, rec.sub, scope, rec.familyId);
602     const out = { access_token: at.token, token_type: 'Bearer',
603       expires_in: TTL.access, scope: scope.join(' ') };
604     if (policy.rotate_refresh) {
605       out.refresh_token = issueRefreshToken(clientId, rec.sub, scope, rec.familyId,
606         rec.generation + 1);
607       log('ok', 'refresh ROTATED to generation ' + (rec.generation + 1));
608     } else {
609       rec.used = false;
610       out.refresh_token = presented;
611       log('warn', 'refresh NOT rotated - the same value keeps being reused');
612     }
613     return json(res, 200, out);
614   }
615
616   // ===== client_credentials: no user, so nobody to consent =====
617   if (grant === 'client_credentials') {
618     const asked = body.get('scope') ? splitScope(body.get('scope')) : client.scopes;
619     const bad = asked.filter((s) => !client.scopes.includes(s));
620     if (bad.length) return json(res, 400, { error: 'invalid_scope',
621       error_description: 'Not registered for: ' + bad.join(' ') });
622     // sub is the CLIENT. Never use this grant to act on a user's behalf.
623     // `resource` (RFC 8707) lets the caller name which API the token is for;

```

The authorization server and IdP

LINES 624–680 OF 850

```

624     // Lab 8 uses it to mint a token this API must refuse.
625     const at = issueAccessToken(clientId, clientId, asked, null,
626                               body.get('resource'));
627     log('ok', 'client_credentials token issued (no user involved)');
628     return json(res, 200, { access_token: at.token, token_type: 'Bearer',
629                           expires_in: TTL.access, scope: asked.join(' ') });
630 }
631
632 // ===== device grant: the agent polls here until the human approves =====
633 if (grant === DEVICE_GRANT) {
634     const entry = devices.get(body.get('device_code'));
635     if (!entry || entry.clientId !== clientId)
636         return json(res, 400, { error: 'invalid_grant' });
637     if (entry.expiresAt < now()) {
638         devices.delete(body.get('device_code'));
639         return json(res, 400, { error: 'expired_token',
640                               error_description: 'The human did not approve in time.' });
641     }
642     // Enforce the polling interval we advertised, so one agent cannot
643     // hammer the token endpoint while it waits.
644     if (now() - entry.lastPoll < 5) {
645         entry.lastPoll = now();
646         return json(res, 400, { error: 'slow_down',
647                               error_description: 'Poll no faster than the advertised interval.' });
648     }
649     entry.lastPoll = now();
650
651     if (entry.status === 'pending') return json(res, 400, {
652         error: 'authorization_pending',
653         error_description: 'Waiting for the human to approve.' });
654     if (entry.status === 'denied') {
655         devices.delete(body.get('device_code'));
656         return json(res, 400, { error: 'access_denied' });
657     }
658
659     devices.delete(body.get('device_code')); // single use, like a code
660     const familyId = C.randomId(8);
661     const at = issueAccessToken(clientId, entry.sub, entry.scope, familyId,
662                               API, { details: entry.details });
663     log('ok', 'device grant completed for ' + entry.sub);
664     const out = { access_token: at.token, token_type: 'Bearer',
665                 expires_in: TTL.access, scope: entry.scope.join(' ') };
666     if (client.grants.includes('refresh_token'))
667         out.refresh_token = issueRefreshToken(clientId, entry.sub, entry.scope,
668                                               familyId, 1);
669     if (entry.details) out.authorization_details = entry.details;
670     return json(res, 200, out);
671 }
672
673 // ===== token exchange (RFC 8693): act on someone else's behalf =====
674 // A service that received a user's token swaps it for a downstream one.
675 // The new token names the caller in `act`, so the API sees the chain.
676 if (grant === EXCHANGE_GRANT) {
677     const subject = body.get('subject_token');
678     const crypto2 = require('node:crypto');
679     const r = C.verifyJwt(subject, {
680         getKey: (kid) => kid === KEY.kid

```

The authorization server and IdP

LINES 681–737 OF 850

```

681     ? crypto2.createPublicKey({ key: KEY.publicJwk, format: 'jwk' }) : null,
682     issuer: ISSUER, requiredTyp: 'at+jwt' });
683   if (!r.valid) return json(res, 400, { error: 'invalid_grant',
684     error_description: 'subject_token is not valid here (' + r.reason + ')'});
685
686   // Delegation may only ever NARROW. You cannot exchange your way up.
687   const held = splitScope(r.payload.scope);
688   const asked = body.get('scope') ? splitScope(body.get('scope')) : held;
689   const up = asked.filter((x) => !held.includes(x));
690   if (up.length) return json(res, 400, { error: 'invalid_scope',
691     error_description: 'Cannot widen scope on exchange: ' + up.join(' ') });
692
693   const parsed = parseDetails(body.get('authorization_details'));
694   if (parsed && parsed.error) return json(res, 400, {
695     error: 'invalid_authorization_details', error_description: parsed.error });
696
697   // Build the actor chain: whoever was acting before is nested inside.
698   const act = { sub: clientId };
699   if (r.payload.act) act.act = r.payload.act;
700   const at = issueAccessToken(clientId, r.payload.sub, asked, null,
701     body.get('resource') || API,
702     { act, details: parsed ? parsed.value : r.payload.authorization_details });
703   log('ok', 'token exchanged: ' + clientId + ' now acting for ' + r.payload.sub);
704   return json(res, 200, { access_token: at.token, token_type: 'Bearer',
705     issued_token_type: ACCESS_TOKEN_TYPE,
706     expires_in: TTL.access, scope: asked.join(' ') });
707 }
708
709 return json(res, 400, { error: 'unsupported_grant_type' });
710 }
711
712 // ----- OIDC: /userinfo -----
713 // Claims about the signed-in user, filtered by the scopes actually granted.
714 // Note what it takes: the ACCESS token, not the ID token. The ID token is
715 // for the client to read; this endpoint is for fetching fresh claims.
716 if (path === '/userinfo') {
717   const header = req.headers.authorization || '';
718   if (!header.toLowerCase().startsWith('bearer '))
719     return json(res, 401, { error: 'invalid_token',
720       error_description: 'Send the access token as a Bearer token.' });
721
722   const crypto3 = require('node:crypto');
723   const r = C.verifyJwt(header.slice(7).trim(), {
724     getKey: (kid) => kid === KEY.kid
725       ? crypto3.createPublicKey({ key: KEY.publicJwk, format: 'jwk' }) : null,
726     issuer: ISSUER, requiredTyp: 'at+jwt' });
727   if (!r.valid) return json(res, 401, { error: 'invalid_token', reason: r.reason });
728
729   const granted = splitScope(r.payload.scope);
730   if (!granted.includes('openid'))
731     return json(res, 403, { error: 'insufficient_scope',
732       error_description: 'The openid scope is what turns an OAuth grant into an '
733         + 'OpenID Connect one. Without it there is no identity to return.' });
734
735   const user = userBySub(r.payload.sub);
736   if (!user) return json(res, 404, { error: 'no_such_subject',
737     error_description: 'A machine token has no user behind it.' });

```

The authorization server and IdP

LINES 738–794 OF 850

```

738
739 // `sub` is always present; everything else is gated on scope, exactly as
740 // the claims in the ID token are.
741 const claims = { sub: user.sub };
742 if (granted.includes('profile')) {
743   claims.name = user.name;
744   claims.preferred_username = usernameOf(user.sub);
745 }
746 if (granted.includes('email')) {
747   claims.email = user.email;
748   claims.email_verified = true;
749 }
750 log('ok', '/userinfo for ' + user.sub + ' -> ' + Object.keys(claims).join(', '));
751 return json(res, 200, claims);
752 }
753
754 // ----- introspection (RFC 7662) -----
755 // Must be protected: it answers questions about other people's tokens.
756 if (path === '/introspect' && req.method === 'POST') {
757   const auth = authenticateClient(req, body);
758   if (auth.error || auth.client.type !== 'confidential')
759     return json(res, 401, { error: 'invalid_client' });
760   const crypto = require('node:crypto');
761   const r = C.verifyJwt(body.get('token'), {
762     getKey: (kid) => kid === KEY.kid
763       ? crypto.createPublicKey({ key: KEY.publicJwk, format: 'jwk' }) : null,
764     issuer: ISSUER });
765   if (!r.valid) return json(res, 200, { active: false, reason: r.reason });
766   if (revoked.has(r.payload.jti)) return json(res, 200, { active: false,
767     reason: 'revoked',
768     note: 'The signature is fine but this jti was revoked before exp. Only '
769       + 'introspection can see that - local JWT validation cannot.' });
770   return json(res, 200, { active: true, scope: r.payload.scope, sub: r.payload.sub,
771     client_id: r.payload.client_id, aud: r.payload.aud, exp: r.payload.exp,
772     jti: r.payload.jti, token_type: 'Bearer' });
773 }
774
775 // ----- revocation (RFC 7009) -----
776 // Always answers 200, even for an unknown token: this endpoint must not
777 // become an oracle for guessing valid ones.
778 if (path === '/revoke' && req.method === 'POST') {
779   const auth = authenticateClient(req, body);
780   if (auth.error) return json(res, 401, { error: auth.error });
781   const token = body.get('token');
782   const rec = refresh.get(C.sha256hex(String(token || '')));
783   if (rec) {
784     const n = revokeFamily(rec.familyId);
785     log('ok', 'revoked refresh family (' + n + ' tokens)');
786     return json(res, 200, { revoked: true, family_tokens: n });
787   }
788   const d = C.decodeJwt(token);
789   if (d && d.payload.jti) {
790     revoked.add(d.payload.jti);
791     log('ok', 'revoked access token jti=' + d.payload.jti);
792     return json(res, 200, { revoked: true,
793       note: 'An API doing local JWT validation will keep accepting this until exp. '
794         + 'That is the trade-off of self-contained tokens.' });

```

The authorization server and IdP

LINES 795–850 OF 850

```

795     }
796     return json(res, 200,
797       { revoked: true, note: 'Unknown token; 200 anyway per RFC 7009.' });
798   }
799
800   return json(res, 404, { error: 'not_found', path });
801 });
802
803 // --- the IdP's two screens -----
804 function resume(res, rid, sid) {
805   const r = requests.get(rid);
806   if (!r) return html(res, '<h2>Expired</h2>', 400);
807   const s = sessions.get(sid);
808   if (!s || s.expiresAt < now()) return loginPage(res, rid, r);
809   return consentPage(res, rid, r, userBySub(s.sub));
810 }
811
812 function loginPage(res, rid, r, error) {
813   return html(res,
814     '<h2>Sign in</h2><p><b>' + esc(r.clientId) + '</b> wants to act on your behalf. '
815     + 'Notice you are typing your password into the <b>identity provider</b> '
816     + '(localhost:7000), never into the application. That is the whole point.</p>'
817     + (error ? '<p style="color:#c0203f"><b>' + esc(error) + '</b></p>' : '')
818     + '<form method=POST action=/login>'
819     + '<input type=hidden name=request_id value="' + esc(rid) + '">'
820     + '<p><label>Username<br><input name=username autofocus></label>'
821     + '<p><label>Password<br><input name=password type=password></label>'
822     + '<p><button>Sign in</button></form>'
823     + '<hr><p style="color:#666;font-size:13px">alice / wonderland &middot; '
824     + 'bob / builder (bob cannot grant payments:write)</p>');
825 }
826
827 function consentPage(res, rid, r, user) {
828   const grantable = r.scope.filter((s) => user.can.includes(s));
829   const refused = r.scope.filter((s) => !user.can.includes(s));
830   return html(res,
831     '<h2>Allow access?</h2><p>Signed in as <b>' + esc(user.name) + '</b>. '
832     + '<b>' + esc(r.clientId) + '</b> is asking for:</p>'
833     + (refused.length ? '<p style="color:#a8620a">' + esc(user.name) + ' cannot '
834       + 'delegate <code>' + esc(refused.join(' ')) + '</code>, so it is dropped. A '
835       + 'client never receives more authority than the user has.</p>' : '')
836     + '<form method=POST action=/consent>'
837     + '<input type=hidden name=request_id value="' + esc(rid) + '">'
838     + grantable.map((s) => '<p><label><input type=checkbox name=scope value="'
839       + esc(s) + '" checked' + (s === 'openid' ? ' disabled' : '') + '> <code>'
840       + esc(s) + '</code></label>').join('')
841     + '<p><button name=action value=allow>Allow</button> '
842     + '<button name=action value=deny>Deny</button></form>');
843 }
844
845 server.listen(PORT, () => {
846   console.log('\n Authorization server + IdP  ' + ISSUER);
847   console.log('  discovery  ' + ISSUER + '/.well-known/oauth-authorization-server');
848   console.log('  policy      ' + ISSUER + '/policy      (turn rules off for Lab 3)');
849   console.log('  signing kid ' + KEY.kid + '\n');
850 });

```

The resource server

The four gates of Chapter 12, in order, in one function. Note that steps 1–3 never contact the authorization server: the signature is checked offline against cached public keys.

LINES 1–51 OF 252

```

1  'use strict';
2  // =====
3  // LAB FILE 3 of 4 – api-server.js          run: node api-server.js
4  // The resource server: the API that owns the data. On every request it runs
5  // the four checks from Chapter 12, and nothing else.
6  // =====
7  const http = require('node:http');
8  const crypto = require('node:crypto');
9  const C = require('./crypto-lab');
10
11  const PORT = 7010;
12  const AUDIENCE = 'http://localhost:7010'; // tokens must name US in `aud`
13  const TRUSTED_ISSUER = 'http://localhost:7000'; // and be signed by THEM
14
15  // The API's own data. The token says WHO is asking and WHAT they may do; the
16  // data belongs to the API, not to the token.
17  const DB = {
18    'u-1001': { profile: { name: 'Alice Liddell', tier: 'gold' }, payments: [],
19              accounts: [{ id: 'ACC-4410', label: 'Everyday Checking', balance: 4820.55 },
20                        { id: 'ACC-9982', label: 'Rainy Day Savings', balance: 15200.0 } ] },
21    'u-1002': { profile: { name: 'Bob Builder', tier: 'standard' }, payments: [],
22              accounts: [{ id: 'ACC-7100', label: 'Joint Account', balance: 310.2 } ] },
23  };
24
25  // --- the JWKS cache -----
26  // Fetch the issuer's public keys once, cache them by kid, and refetch when an
27  // unknown kid appears. That is how key rotation works with no shared secret
28  // and no downtime. Restarting the AS rotates its key – watch this recover.
29  const jwks = { keys: new Map(), fetchedAt: 0 };
30
31  async function refreshJwks(why) {
32    const r = await fetch(TRUSTED_ISSUER + '/jwks.json');
33    const body = await r.json();
34    jwks.keys.clear();
35    for (const jwk of body.keys)
36      jwks.keys.set(jwk.kid, crypto.createPublicKey({ key: jwk, format: 'jwk' }));
37    jwks.fetchedAt = Date.now();
38    log('ok', 'JWKS fetched (' + why + '): ' + [...jwks.keys.keys()].join(', '));
39  }
40
41  async function keyFor(kid) {
42    if (!jwks.keys.size) await refreshJwks('cold start');
43    if (kid && !jwks.keys.has(kid)) await refreshJwks('unknown kid ' + kid);
44    return jwks.keys.get(kid) || null;
45  }
46
47  const log = (tag, msg) => console.log(' [api] ' + tag.padEnd(7) + msg);
48
49  function readBody(req) {
50    return new Promise((resolve) => {
51      let raw = '';

```

The resource server

LINES 52-102 OF 252

```

52     req.on('data', (c) => { raw += c; if (raw.length > 1e5) req.destroy(); });
53     req.on('end', () => resolve(new URLSearchParams(raw)));
54   });
55 }
56
57 // Flatten the RFC 8693 `act` chain into something loggable:
58 //   alice <- demo-orchestrator <- demo-agent
59 function actorChain(claims) {
60   const chain = [];
61   for (let a = claims.act; a; a = a.act) chain.push(a.sub);
62   return chain;
63 }
64
65 // Find the structured grant (RFC 9396) that covers a given action.
66 function detailFor(claims, type) {
67   return (claims.authorization_details || []).find((d) => d.type === type) || null;
68 }
69 const json = (res, status, obj) => (res.writeHead(status,
70   { 'content-type': 'application/json' }), res.end(JSON.stringify(obj, null, 2)));
71
72 // =====
73 // The four checks, in order. This function IS the resource server.
74 // =====
75 async function authorize(req, res, required) {
76   // 1. Is there a Bearer token?                                -> 401
77   const header = req.headers.authorization || '';
78   if (!header.toLowerCase().startsWith('bearer ')) {
79     res.setHeader('www-authenticate',
80       'Bearer realm="" + AUDIENCE + "", scope="" + required.join(' ') + "");
81     log('401', 'no Bearer token');
82     return { body: { error: 'invalid_token',
83       error_description: 'This endpoint needs an OAuth 2.1 Bearer access token.' },
84       status: 401 };
85   }
86   const token = header.slice(7).trim();
87
88   // 2. Is the signature real and mine to trust?                -> 401
89   // 3. Was it minted for THIS api, and is it still live? -> 401
90   const decoded = C.decodeJwt(token);
91   const kid = decoded && decoded.header ? decoded.header.kid : null;
92   const key = await keyFor(kid);
93   let result = C.verifyJwt(token, { getKey: () => key, issuer: TRUSTED_ISSUER,
94     audience: AUDIENCE, requiredTyp: 'at+jwt' });
95
96   // A signature failure on a kid we thought we knew usually means the key
97   // behind it changed. Refetch once before blaming the caller.
98   if (!result.valid && result.reason === 'bad_signature'
99     && Date.now() - jwks.fetchedAt > 5000) {
100     await refreshJwks('signature failure on a known kid');
101     result = C.verifyJwt(token, { getKey: () => jwks.keys.get(kid) || null,
102       issuer: TRUSTED_ISSUER, audience: AUDIENCE, requiredTyp: 'at+jwt' });

```

The resource server

LINES 103–152 OF 252

```

103   }
104
105   if (!result.valid) {
106     res.setHeader('www-authenticate', 'Bearer error="invalid_token"');
107     log('401', result.reason);
108     return { status: 401, body: { error: 'invalid_token', check_failed: result.reason,
109       error_description: {
110         unexpected_typ: 'Expected typ="at+jwt". You almost certainly sent the ID '
111           + 'TOKEN instead of the access token – a very common bug.',
112         audience_mismatch: 'This token was minted for another audience. A perfect '
113           + 'signature does not make a token valid HERE.',
114         unsupported_alg: 'Only RS256 is accepted. Never trust the token\'s own alg.',
115         unknown_kid: 'No published key matches. The AS may have rotated keys.',
116         bad_signature: 'Altered, or signed by something other than the issuer.',
117         token_expired: 'Access tokens are short-lived on purpose. Refresh it.',
118         issuer_mismatch: 'This API only trusts ' + TRUSTED_ISSUER + '.',
119       }[result.reason] || result.reason } } };
120   }
121
122   // 4. Does the scope cover THIS operation?           -> 403, not 401
123   const granted = String(result.payload.scope || '').split(/\s+/).filter(Boolean);
124   const missing = required.filter((s) => !granted.includes(s));
125   if (missing.length) {
126     res.setHeader('www-authenticate', 'Bearer error="insufficient_scope", scope="'
127       + required.join(' ') + '"');
128     log('403', 'insufficient_scope, needs ' + missing.join(' '));
129     return { status: 403, body: { error: 'insufficient_scope',
130       required_scope: required, granted_scope: granted,
131       error_description: 'The token is perfectly valid – it simply does not '
132         + 'authorize this action. That is why this is 403 and not 401: refreshing '
133         + 'will not help, the user has to grant more.' } } };
134   }
135
136   log('ok', req.method + ' ' + req.url + ' sub=' + result.payload.sub
137     + ' scope="' + result.payload.scope + '"');
138   return { claims: result.payload };
139 }
140
141 // =====
142 const server = http.createServer(async (req, res) => {
143   const url = new URL(req.url, AUDIENCE);
144   const path = url.pathname;
145
146   if (path === '/healthz') return json(res, 200,
147     { ok: true, audience: AUDIENCE, cached_kids: [...jwks.keys.keys()] });
148
149   // ---- profile, needs `profile` ----
150   if (path === '/api/me') {
151     const a = await authorize(req, res, ['profile']);
152     if (a.status) return json(res, a.status, a.body);

```

The resource server

LINES 153–202 OF 252

```

153     const rec = DB[a.claims.sub];
154     // A client_credentials token has a client_id as its sub, and no human
155     // behind it - so there is no profile to return.
156     if (!rec) return json(res, 404, { error: 'no_such_subject', sub: a.claims.sub,
157       note: 'Valid token, but sub is not a user. This is what a machine token '
158       + 'looks like here.' });
159     return json(res, 200, { sub: a.claims.sub, ...rec.profile });
160   }
161
162   // ---- accounts, needs `accounts:read` ----
163   if (path === '/api/accounts') {
164     const a = await authorize(req, res, ['accounts:read']);
165     if (a.status) return json(res, a.status, a.body);
166     const rec = DB[a.claims.sub];
167     return json(res, 200, { sub: a.claims.sub, accounts: rec ? rec.accounts : [],
168       acting_chain: actorChain(a.claims),
169       note: 'The API picked the data from the token\'s `sub`, never from a user id '
170       + 'in the request. Trusting a caller-supplied id here is how you build an '
171       + 'IDOR on top of perfectly good OAuth.' });
172   }
173
174   // ---- payments, needs `payments:write` ----
175   // Scope says the caller may move money at all. authorization_details says
176   // how much, and out of which account. See Chapter 15.
177   if (path === '/api/payments' && req.method === 'POST') {
178     const body = await readBody(req);
179     const a = await authorize(req, res, ['payments:write']);
180     if (a.status) return json(res, a.status, a.body);
181     const rec = DB[a.claims.sub];
182     if (!rec) return json(res, 404, { error: 'no_such_subject' });
183
184     const amount = Number(body.get('amount') || 42);
185     const from = body.get('from') || rec.accounts[0].id;
186     const chain = actorChain(a.claims);
187     if (chain.length)
188       log('info', 'acting chain: ' + a.claims.sub + ' <- ' + chain.join(' <- '));
189
190     // If the token carries a structured grant, it BINDS the request. A scope
191     // alone would have allowed any amount from any account.
192     const grant = detailFor(a.claims, 'payment_initiation');
193     if (grant) {
194       const cap = Number(grant.maxAmount);
195       if (Number.isFinite(cap) && amount > cap) {
196         log('403', 'over the granted limit: ' + amount + ' > ' + cap);
197         return json(res, 403, { error: 'insufficient_authorization',
198           error_description: 'This token authorizes at most ' + grant.maxAmount
199           + ' per payment; the request asked for ' + amount + '. The scope '
200           + 'payments:write was satisfied - the structured limit was not.',
201           granted: grant, requested: { amount, from } });
202       }

```

The resource server

LINES 203–252 OF 252

```

203     if (grant.fromAccount && grant.fromAccount !== from) {
204         log('403', 'wrong source account: ' + from);
205         return json(res, 403, { error: 'insufficient_authorization',
206             error_description: 'This token only authorizes payments from '
207                 + grant.fromAccount + '.', granted: grant, requested: { amount, from } });
208     }
209 }
210
211 const payment = { id: 'PAY-' + Math.random().toString(36).slice(2, 8).toUpperCase(),
212     from, to: body.get('to') || 'ACC-EXTERNAL', amount, status: 'accepted',
213     authorized_by: a.claims.sub, acting: chain.length ? chain : undefined,
214     constrained_by: grant || undefined };
215 rec.payments.push(payment);
216 log('ok', 'payment ' + payment.id + ' for ' + amount);
217 return json(res, 201, { payment });
218 }
219
220 // ---- the same data, authorized by introspection instead (Lab 8) ----
221 // Slower - a network call per request - but it sees revocation instantly.
222 if (path === '/api/accounts-introspected') {
223     const header = req.headers.authorization || '';
224     const token = header.toLowerCase().startsWith('bearer ') ? header.slice(7) : '';
225     const r = await fetch(TRUSTED_ISSUER + '/introspect', { method: 'POST',
226         headers: { 'content-type': 'application/x-www-form-urlencoded',
227             authorization: 'Basic '
228                 + Buffer.from('demo-service:service-secret').toString('base64') },
229         body: new URLSearchParams({ token }).toString() });
230     const intro = await r.json();
231     if (!intro.active) {
232         log('401', 'introspection says not active: ' + intro.reason);
233         return json(res, 401, { error: 'invalid_token', introspection: intro,
234             note: 'Local JWT validation would still have accepted a revoked-but-'
235                 + 'unexpired token. Introspection is the only way an API learns about '
236                 + 'revocation immediately - at the cost of a round trip per call.' });
237     }
238     const rec = DB[intro.sub];
239     log('ok', 'introspection says active, sub=' + intro.sub);
240     return json(res, 200, { sub: intro.sub, accounts: rec ? rec.accounts : [],
241         validated_by: 'token introspection (RFC 7662)' });
242 }
243
244 return json(res, 404, { error: 'not_found', path });
245 });
246
247 server.listen(PORT, () => {
248     console.log('\n Resource server ' + AUDIENCE);
249     console.log(' trusts issuer ' + TRUSTED_ISSUER);
250     console.log(' scopes /api/me=profile /api/accounts=accounts:read'
251         + ' /api/payments=payments:write\n');
252 });

```

The client, and all fifteen labs

The client walks the front channel with a cookie jar instead of a browser, so every lab is one command whose result is printed rather than clicked. Each LABS.* entry is one exercise.

LINES 1–53 OF 705

```

1  'use strict';
2  // =====
3  // LAB FILE 4 of 4 – client.js           run: node client.js <lab>
4  // The OAuth 2.1 client, plus every lab scenario. It walks the front channel
5  // with a cookie jar instead of a browser, so each lab is one command and the
6  // result is printed, not clicked.
7  //
8  //   node client.js list                 show every lab
9  //   node client.js happy               Lab 1 the correct flow
10 //   node client.js all                 run all of them
11 // =====
12 const C = require('./crypto-lab');
13
14 const AS = 'http://localhost:7000';
15 const API = 'http://localhost:7010';
16 const REDIRECT = 'http://localhost:7020/callback';
17 const CLIENT = { id: 'demo-web-app', secret: 'web-app-secret' };
18 const SPA = { id: 'demo-spa', secret: null };
19 const SERVICE = { id: 'demo-service', secret: 'service-secret' };
20 const AGENT = { id: 'demo-agent', secret: null };
21 const ORCH = { id: 'demo-orchestrator', secret: 'orchestrator-secret' };
22 const DEVICE_GRANT = 'urn:ietf:params:oauth:grant-type:device_code';
23 const EXCHANGE_GRANT = 'urn:ietf:params:oauth:grant-type:token-exchange';
24
25 // --- printing -----
26 const B = '\x1b[1m', D = '\x1b[2m', R = '\x1b[0m';
27 const CY = '\x1b[36m', GR = '\x1b[32m', RE = '\x1b[31m', YE = '\x1b[33m';
28 const step = (n, s) => console.log('\n' + CY + B + 'STEP ' + n + ' ' + s + R);
29 const why = (s) => console.log(' ' + D + s + R);
30 const val = (k, v) => console.log(' ' + k.padEnd(20) + ' ' + v);
31 const pass = (s) => console.log(' ' + GR + B + 'PASS' + R + ' ' + s);
32 const fail = (s) => console.log(' ' + RE + B + 'FAIL' + R + ' ' + s);
33 const note = (s) => console.log(' ' + YE + B + 'NOTE' + R + ' ' + s);
34 const head = (s) => console.log('\n' + B + '='.repeat(70) + '\n' + s + '\n'
35   + '='.repeat(70) + R);
36
37 // --- a browser, more or less -----
38 // Keeps one cookie and never follows redirects, so we can read the Location
39 // header the way the browser's address bar would show it.
40 function makeBrowser() {
41   let cookie = null;
42   return async function go(url, options = {}) {
43     const headers = { ...(options.headers || {}) };
44     if (cookie) headers.cookie = cookie;
45     const res = await fetch(url, { ...options, headers, redirect: 'manual' });
46     const set = res.headers.get('set-cookie');
47     if (set) cookie = set.split(';')[0];
48     return { status: res.status, location: res.headers.get('location'),
49       body: await res.text() };
50   };
51 }
52
53 const form = (obj) => ({ method: 'POST', body: new URLSearchParams(obj).toString(),

```

The client, and all fifteen labs

LINES 54–108 OF 705

```

54   headers: { 'content-type': 'application/x-www-form-urlencoded' } });
55
56 // --- the token endpoint -----
57 async function token(params, client) {
58   const headers = { 'content-type': 'application/x-www-form-urlencoded' };
59   const body = new URLSearchParams(params);
60   if (client.secret) {
61     // A confidential client authenticates. RFC 6749 2.3.1 wants the id and
62     // secret form-urlencoded before base64.
63     headers.authorization = 'Basic ' + Buffer.from(
64       encodeURIComponent(client.id) + ':' + encodeURIComponent(client.secret)
65     ).toString('base64');
66   } else {
67     body.set('client_id', client.id); // public: identified, not authenticated
68   }
69   const res = await fetch(AS + '/token', { method: 'POST', headers,
70     body: body.toString() });
71   return { status: res.status, body: await res.json() };
72 }
73
74 async function callApi(path, accessToken, method = 'GET') {
75   const headers = {};
76   if (accessToken) headers.authorization = 'Bearer ' + accessToken;
77   const res = await fetch(API + path, { method, headers });
78   return { status: res.status, body: await res.json() };
79 }
80
81 const setPolicy = (key, value) =>
82   fetch(AS + '/policy', form({ key, value: String(value) })).then((r) => r.json());
83
84 // POST a form to any AS endpoint, authenticating as the given client.
85 async function post(path, params, client) {
86   const headers = { 'content-type': 'application/x-www-form-urlencoded' };
87   const body = new URLSearchParams(params);
88   if (client && client.secret) {
89     headers.authorization = 'Basic ' + Buffer.from(
90       encodeURIComponent(client.id) + ':' + encodeURIComponent(client.secret)
91     ).toString('base64');
92   } else if (client) { body.set('client_id', client.id); }
93   const res = await fetch(AS + path, { method: 'POST', headers,
94     body: body.toString() });
95   return { status: res.status, body: await res.json().catch(() => ({})) };
96 }
97 const pause = (ms) => new Promise((r) => setTimeout(r, ms));
98
99 // =====
100 // The front channel, walked end to end. Returns the authorization code.
101 // =====
102 async function getCode({ client = CLIENT, scope = 'openid profile accounts:read',
103   user = 'alice', password = 'wonderland',
104   pkce = true, redirect = REDIRECT, quiet = false } = {}) {
105   const browser = makeBrowser();
106   const verifier = C.createVerifier();
107   const challenge = C.challengeS256(verifier);
108   const state = C.randomToken(16);

```

The client, and all fifteen labs

LINES 109–163 OF 705

```

109   const nonce = C.randomToken(12);
110
111   const q = new URLSearchParams({ response_type: 'code', client_id: client.id,
112     redirect_uri: redirect, scope, state, nonce });
113   if (pkce) { q.set('code_challenge', challenge);
114     q.set('code_challenge_method', 'S256'); }
115
116   if (!quiet) {
117     val('code_verifier', verifier);
118     why('43 random chars. Stays here; never crosses the browser.');
```

val('code_challenge', pkce ? challenge : '(none - PKCE skipped)');

```

119     if (pkce) why('= BASE64URL(SHA256(verifier)). Safe to put in a URL.');
```

}

```

122
123   // The browser follows the redirect to /authorize and lands on a login form.
124   let r = await browser(AS + '/authorize?' + q.toString());
125   if (r.status === 302 && r.location && r.location.includes('error=')) {
126     const u = new URL(r.location);
127     return { error: u.searchParams.get('error'),
128       description: u.searchParams.get('error_description'), verifier };
129   }
130   const rid = (r.body.match(/name=request_id value="([^"]+)"/) || [])[1];
131   if (!rid) return { error: 'no_login_form', description: r.body.slice(0, 200) };
132
133   // The password is typed into the IdP's own origin, not into this app.
134   r = await browser(AS + '/login', form({ request_id: rid, username: user, password }));
135   if (r.status === 302) r = await browser(AS + r.location.replace(AS, ''));
136   const offered = [...r.body.matchAll(/name=scope value="([^"]+)"/g)].map((m) => m[1]);
137
138   // Approve everything the consent screen actually offered.
139   const consent = new URLSearchParams({ request_id: rid, action: 'allow' });
140   for (const s of offered) consent.append('scope', s);
141   r = await browser(AS + '/consent', { method: 'POST', body: consent.toString(),
142     headers: { 'content-type': 'application/x-www-form-urlencoded' } });
143
144   const u = new URL(r.location);
145   if (u.searchParams.get('error'))
146     return { error: u.searchParams.get('error'),
147       description: u.searchParams.get('error_description'), verifier };
148   return { code: u.searchParams.get('code'), state: u.searchParams.get('state'),
149     iss: u.searchParams.get('iss'), verifier, challenge, sentState: state,
150     sentNonce: nonce, offered };
151 }
152
153 // =====
154 // The labs
155 // =====
156 const LABS = {};
157
158 // ----- Lab 1
159 LABS.happy = { title: 'The correct flow, end to end', run: async () => {
160   step(1, 'Create the PKCE pair, then walk the front channel');
161   const f = await getCode();
162   if (f.error) return fail(f.error + ': ' + f.description);
163   val('code', f.code);

```

The client, and all fifteen labs

LINES 164–218 OF 705

```

164     val('state echoed', f.state);
165     val('iss (RFC 9207)', f.iss);
166     f.state === f.sentState ? pass('state matches - this callback is ours')
167       : fail('state mismatch');
168
169     step(2, 'Back channel: POST /token with the code AND the verifier');
170     why('The server re-hashes our verifier and compares it to the challenge it');
171     why('stored in step 1. Only the app that made the verifier can pass.');
```

172 const t = await token({ grant_type: 'authorization_code', code: f.code,

173 redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);

174 if (t.status !== 200) return fail(JSON.stringify(t.body));

175 pass('tokens issued');

176 val('token_type', t.body.token_type);

177 val('expires_in', t.body.expires_in + 's');

178 val('scope', t.body.scope);

179

180 step(3, 'What is actually inside the access token');

181 console.log(' ' + JSON.stringify(C.decodeJwt(t.body.access_token).header));

182 console.log(JSON.stringify(C.decodeJwt(t.body.access_token).payload, null, 2)

183 .split('\n').map((l) => ' ' + l).join('\n'));

184 why('Note aud (which API may accept it), scope, and exp.');

185

186 step(4, 'Call the API');

187 const a = await callApi('/api/accounts', t.body.access_token);

188 a.status === 200 ? pass('200 - ' + a.body.accounts.length + ' accounts')

189 : fail(JSON.stringify(a.body));

190

191 step(5, 'Ask for something the token is not scoped for');

192 const p = await callApi('/api/payments', t.body.access_token, 'POST');

193 p.status === 403 ? pass('403 insufficient_scope - the API enforced scope, not '

194 + 'just a valid signature')

195 : fail('expected 403, got ' + p.status);

196 why(p.body.error_description || '');

197 return t.body;

198 } };

199

200 // ----- Lab 2

201 LABS.pkce = { title: 'The PKCE transform, step by step', run: async () => {

202 const v = C.createVerifier();

203 const e = C.explainPkce(v);

204 step(1, 'One random string, one hash');

205 val('code_verifier', e.code_verifier);

206 val('length', e.length + ' characters');

207 step(2, 'SHA-256 of the verifier TEXT');

208 val('sha256 (hex)', e.sha256_hex);

209 val('sha256 (base64)', e.sha256_base64);

210 val('code_challenge', e.code_challenge);

211 why('base64url is base64 with + -> -, / -> _, and the = padding removed.');

212 step(3, 'What the server does with them');

213 pass('correct verifier -> ' + C.verifyPkce(v, e.code_challenge, 'S256').ok);

214 const wrong = C.verifyPkce(C.createVerifier(), e.code_challenge, 'S256');

215 pass('a different one -> ' + wrong.ok + ' (' + wrong.reason + ')');

216 why('The challenge is a hash, so publishing it gives an attacker nothing:');

217 why('reversing it means computing a SHA-256 preimage.');

218 } };

The client, and all fifteen labs

LINES 219–273 OF 705

```

219
220 // ----- Lab 3
221 LABS.attack = { title: 'A stolen authorization code, refused by PKCE',
222   run: async () => {
223     step(1, 'A perfectly normal flow – and the code leaks');
224     const f = await getCode({ client: SPA, scope: 'openid accounts:read',
225       redirect: 'http://localhost:7020/spa-callback' });
226     if (f.error) return fail(f.error + ': ' + f.description);
227     val('stolen code', f.code);
228     why('Assume the worst: a malicious app, a leaked log, a crafted redirect.');
```

229 why('This is the PUBLIC client, so there is no secret to stop the attacker.');

```

230
231     step(2, 'The attacker redeems it – but has no code_verifier');
232     const a1 = await token({ grant_type: 'authorization_code', code: f.code,
233       redirect_uri: 'http://localhost:7020/spa-callback' }, SPA);
234     a1.status !== 200 ? pass('REFUSED: ' + a1.body.error)
235       : fail('the attack SUCCEEDED – is require_pkce off?');
236     why(a1.body.error_description || '');
237
238     step(3, 'The attacker guesses a verifier');
239     const a2 = await token({ grant_type: 'authorization_code', code: f.code,
240       redirect_uri: 'http://localhost:7020/spa-callback',
241       code_verifier: C.createVerifier() }, SPA);
242     a2.status !== 200 ? pass('REFUSED: the hash of a guess never matches')
243       : fail('unexpected success');
```

244 note('The verifier is 256 bits of randomness. Guessing it is not an attack,');

245 note('it is a lottery with no winning ticket. THAT is what PKCE buys you.');

```

246   } };
247
248 // ----- Lab 4
249 LABS.nopkce = { title: 'The same theft with PKCE turned OFF', run: async () => {
250   note('Turning require_pkce off. This is the pre-2013 world.');
```

251 await setPolicy('require_pkce', false);

```

252   step(1, 'Get a code with no code_challenge at all');
253   const f = await getCode({ client: SPA, scope: 'openid accounts:read',
254     redirect: 'http://localhost:7020/spa-callback', pkce: false });
255   if (f.error) { await setPolicy('require_pkce', true);
256     return fail(f.error + ': ' + f.description); }
257   val('stolen code', f.code);
258
259   step(2, 'The attacker redeems it with nothing but the code');
```

260 const a = await token({ grant_type: 'authorization_code', code: f.code,

261 redirect_uri: 'http://localhost:7020/spa-callback' }, SPA);

```

262   if (a.status === 200) {
263     fail('ATTACK SUCCEEDED – the attacker now holds a real access token');
```

264 val('access_token', a.body.access_token.slice(0, 40) + '...');

```

265     const d = await callApi('/api/accounts', a.body.access_token);
266     if (d.status === 200) fail('and it WORKS: ' + JSON.stringify(d.body.accounts[0]));
267     note('The client_id is public, so the code alone was enough. This is the');
```

268 note('exact vulnerability OAuth 2.1 closes by making PKCE mandatory.');

```

269   } else { pass('refused: ' + a.body.error); }
270
271   step(3, 'Restore the safe default');
```

272 await setPolicy('require_pkce', true);

```

273   pass('require_pkce is back on');
```

The client, and all fifteen labs

LINES 274–327 OF 705

```

274 } };
275
276 // ----- Lab 5
277 LABS.replay = { title: 'Replaying an already-used code', run: async () => {
278   const f = await getCode({ quiet: true });
279   step(1, 'First redemption (legitimate)');
280   const a = await token({ grant_type: 'authorization_code', code: f.code,
281     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
282   a.status === 200 ? pass('tokens issued') : fail(JSON.stringify(a.body));
283   step(2, 'Second redemption of the same code, same verifier');
284   const b = await token({ grant_type: 'authorization_code', code: f.code,
285     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
286   b.status !== 200 ? pass('REFUSED: codes are single-use')
287     : fail('replay accepted');
288   why(b.body.error_description || '');
289 } };
290
291 // ----- Lab 6
292 LABS.refresh = { title: 'Refresh rotation and reuse detection', run: async () => {
293   const f = await getCode({ quiet: true });
294   const first = await token({ grant_type: 'authorization_code', code: f.code,
295     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
296   const rt1 = first.body.refresh_token;
297   val('refresh gen 1', rt1.slice(0, 24) + '...');
298
299   step(1, 'Refresh once');
300   const second = await token({ grant_type: 'refresh_token', refresh_token: rt1 },
301     CLIENT);
302   const rt2 = second.body.refresh_token;
303   rt2 && rt2 !== rt1 ? pass('ROTATED - a brand-new refresh token came back')
304     : fail('no rotation');
305   val('refresh gen 2', String(rt2).slice(0, 24) + '...');
306   why('The old value is now retired. Each refresh token is single-use.');
```

```

307
308   step(2, 'Replay the RETIRED generation 1, as a thief would');
309   const replay = await token({ grant_type: 'refresh_token', refresh_token: rt1 },
310     CLIENT);
311   replay.status !== 200 ? pass('REFUSED and the whole family revoked')
312     : fail('reuse accepted');
313   why(replay.body.error_description || '');
314
315   step(3, 'The legitimate generation 2 is collateral damage');
316   const after = await token({ grant_type: 'refresh_token', refresh_token: rt2 },
317     CLIENT);
318   after.status !== 200 ? pass('also refused - the server cannot tell thief from '
319     + 'victim, so it trusts neither')
320     : fail('gen 2 still works');
321   note('The user signs in again. That is the correct, safe outcome: a silent');
322   note('permanent compromise became a visible, recoverable one.');
```

```

323 } };
324
325 // ----- Lab 7
326 LABS.scope = { title: 'Scope is capped three times over', run: async () => {
327   step(1, 'Sign in as bob and ask for payments:write');
```

The client, and all fifteen labs

LINES 328–381 OF 705

```

328 why('bob is not entitled to payments:write, so consent cannot offer it.');
```

```

329 const f = await getCode({ user: 'bob', password: 'builder',
330   scope: 'openid profile accounts:read payments:write', quiet: true });
331 if (f.error) return fail(f.error + ': ' + f.description);
332 val('requested', 'openid profile accounts:read payments:write');
333 val('offered at consent', f.offered.join(' '));
334 const t = await token({ grant_type: 'authorization_code', code: f.code,
335   redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
336 val('actually granted', t.body.scope);
337 !t.body.scope.includes('payments:write')
338   ? pass('payments:write was dropped - the user never had it to give')
339   : fail('scope escalation!');
```

```

340
341 step(2, 'Use what bob DOES have');
```

```

342 const a = await callApi('/api/accounts', t.body.access_token);
343 a.status === 200 ? pass('200 on /api/accounts') : fail(JSON.stringify(a.body));
344
345 step(3, 'Now try to move money');
```

```

346 const p = await callApi('/api/payments', t.body.access_token, 'POST');
347 p.status === 403 ? pass('403 insufficient_scope') : fail('expected 403');
348 why('A valid token without the scope is 403, never 401. Refreshing will not!');
349 why('help - the user has to grant more.');
```

```

350 } };
351
352 // ----- Lab 8
353 LABS.idtoken = { title: 'An ID token is not an access token', run: async () => {
354   const f = await getCode({ quiet: true });
355   const t = await token({ grant_type: 'authorization_code', code: f.code,
356     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
357
358   step(1, 'Compare the two tokens this one flow returned');
359   const at = C.decodeJwt(t.body.access_token), it = C.decodeJwt(t.body.id_token);
360   val('access typ / aud', at.header.typ + ' / ' + at.payload.aud);
361   val('id typ / aud', it.header.typ + ' / ' + it.payload.aud);
362   why('The access token names the API. The ID token names the CLIENT.');
```

```

363
364   step(2, 'Send the ID token to the API as a Bearer token');
```

```

365   const bad = await callApi('/api/accounts', t.body.id_token);
366   bad.status === 401 ? pass('401 - refused, as it must be') : fail('ACCEPTED!');
367   val('check_failed', bad.body.check_failed);
368   why(bad.body.error_description || '');
369
370   step(3, 'Now a token minted for a DIFFERENT API (RFC 8707 `resource`)');
```

```

371   const m2m = await token({ grant_type: 'client_credentials',
372     scope: 'accounts:read', resource: 'http://localhost:9999' }, SERVICE);
373   const foreign = C.decodeJwt(m2m.body.access_token);
374   val('its aud', foreign.payload.aud);
375   why('Genuine signature, not expired, real issuer. The only thing wrong is aud.');
```

```

376   const rej = await callApi('/api/accounts', m2m.body.access_token);
377   rej.status === 401 ? pass('401 ' + rej.body.check_failed + ' - refused')
378     : fail('accepted a token for another API!');
379   why(rej.body.error_description || '');
380   note('Without the aud check, any API trusting this issuer would accept any');
381   note('token it ever minted. The audience is what confines a token to one API.');
```

The client, and all fifteen labs

LINES 382–435 OF 705

```

382 } };
383
384 // ----- Lab 9
385 LABS.m2m = { title: 'Machine to machine: no user, no consent', run: async () => {
386   step(1, 'client_credentials');
387   const t = await token({ grant_type: 'client_credentials',
388     scope: 'accounts:read profile' }, SERVICE);
389   t.status === 200 ? pass('token issued with no browser and no consent screen')
390     : fail(JSON.stringify(t.body));
391   const d = C.decodeJwt(t.body.access_token);
392   val('sub', d.payload.sub);
393   val('refresh_token', t.body.refresh_token ? 'present' : 'none (correct)');
394   why('sub is the client_id: there is no human here. No refresh token either,');
395   why('because the client can simply ask again with its own credentials.');
```

396

```

397   step(2, 'Ask the API who the user is');
398   const me = await callApi('/api/me', t.body.access_token);
399   me.status === 404 ? pass('404 no_such_subject - correct, nobody delegated')
400     : note('status ' + me.status);
401   note('Never use this grant to act on a user\'s behalf. Nobody consented.');
```

402 } };

403

404 // ----- Lab 10

```

405 LABS.revoke = { title: 'Revocation: local validation vs introspection',
406   run: async () => {
407     const f = await getCode({ scope: 'openid accounts:read', quiet: true });
408     const t = await token({ grant_type: 'authorization_code', code: f.code,
409       redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
410     const at = t.body.access_token;
```

411

```

412     step(1, 'Both endpoints work while the token is live');
413     val('local validation', (await callApi('/api/accounts', at)).status);
414     val('introspection', (await callApi('/api/accounts-introspected', at)).status);
```

415

```

416     step(2, 'Revoke the access token (RFC 7009)');
417     const r = await fetch(AS + '/revoke', { method: 'POST',
418       headers: { 'content-type': 'application/x-www-form-urlencoded',
419         authorization: 'Basic ' + Buffer.from(CLIENT.id + ':' + CLIENT.secret)
420           .toString('base64') },
421       body: new URLSearchParams({ token: at }).toString() });
422     pass('revoked: ' + JSON.stringify(await r.json()));
```

423

```

424     step(3, 'Call both endpoints again');
425     const local = await callApi('/api/accounts', at);
426     const intro = await callApi('/api/accounts-introspected', at);
427     val('local validation', local.status + (local.status === 200
428       ? ' <- STILL ACCEPTED' : ''));
429     val('introspection', intro.status + (intro.status === 401
430       ? ' <- correctly refused' : ''));
431     note('A self-contained JWT knows nothing about being revoked: its signature');
432     note('and exp are unchanged. Introspection asks the server and sees the');
433     note('truth, at the cost of a round trip per call. THAT is why access tokens');
434     note('are given lifetimes measured in minutes.');
```

435 } };

The client, and all fifteen labs

LINES 436–489 OF 705

```

436
437 // ----- Lab 11
438 LABS.oidc = { title: 'Who is this? (OpenID Connect)', run: async () => {
439   step(1, 'Ask for the openid scope, and send a nonce');
440   why('That one scope is what turns an OAuth grant into an OpenID Connect one.');
```

441 const f = await getCode({ scope: 'openid profile email accounts:read' });

442 if (f.error) return fail(f.error + ': ' + f.description);

443 val('nonce we sent', f.sentNonce);

444 const t = await token({ grant_type: 'authorization_code', code: f.code,

445 redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);

446 if (t.status !== 200) return fail(JSON.stringify(t.body));

447 t.body.id_token ? pass('the response now carries an id_token as well')

448 : fail('no id_token – was the openid scope granted?');

449

450 step(2, 'Two tokens, two different jobs');

451 const at = C.decodeJwt(t.body.access_token), it = C.decodeJwt(t.body.id_token);

452 val('access typ / aud', at.header.typ + ' / ' + at.payload.aud);

453 val('id typ / aud', it.header.typ + ' / ' + it.payload.aud);

454 why('The access token is addressed to the API. The ID token is addressed to');

455 why('THIS CLIENT – which is why the client is the one that validates it.');

456 console.log(JSON.stringify(it.payload, null, 2).split('\n')

457 .map((l) => ' ' + l).join('\n'));

458

459 step(3, 'Validate the ID token the way a client must');

460 const jwks = await (await fetch(AS + '/jwks.json')).json();

461 const crypto = require('node:crypto');

462 const key = crypto.createPublicKey({ key: jwks.keys[0], format: 'jwk' });

463 // signature + iss + aud + exp, all in one pass

464 const v = C.verifyJwt(t.body.id_token, { getKey: () => key, issuer: AS,

465 audience: CLIENT.id, requiredTyp: 'JWT' });

466 v.valid ? pass('signature, iss, aud and exp all check out')

467 : fail('rejected: ' + v.reason);

468 // the nonce is the check people forget: it ties this token to THIS request

469 it.payload.nonce === f.sentNonce

470 ? pass('nonce matches the one we generated – not a replayed token')

471 : fail('nonce mismatch');

472 val('sub', it.payload.sub + ' <- the stable identifier. Key your user on this.');

473 val('auth_time', it.payload.auth_time + ' (when they actually authenticated)');

474

475 step(4, 'A token minted for someone else must be refused');

476 const other = C.verifyJwt(t.body.id_token, { getKey: () => key, issuer: AS,

477 audience: 'some-other-app', requiredTyp: 'JWT' });

478 !other.valid ? pass('REFUSED: ' + other.reason + ' – aud is checked, not assumed')

479 : fail('accepted a token addressed to another client!');

480 why('Without this check, any client of this issuer could replay an ID token');

481 why('at any other client and be logged in as that user.');

482

483 step(5, 'Fetch fresh claims from /userinfo');

484 why('Note which token it takes: the ACCESS token, not the ID token.');

485 let r = await fetch(AS + '/userinfo',

486 { headers: { authorization: 'Bearer ' + t.body.access_token } });

487 let b = await r.json();

488 r.status === 200 ? pass('200 ' + JSON.stringify(b)) : fail(JSON.stringify(b));

489 why('The claims are filtered by granted scope, exactly as in the ID token.');

The client, and all fifteen labs

LINES 490–543 OF 705

```

490
491     step(6, 'And without the openid scope there is no identity to return');
492     const f2 = await getCode({ scope: 'accounts:read', quiet: true });
493     const t2 = await token({ grant_type: 'authorization_code', code: f2.code,
494         redirect_uri: REDIRECT, code_verifier: f2.verifier }, CLIENT);
495     val('id_token present', t2.body.id_token ? 'yes' : 'no <- correct');
496     r = await fetch(AS + '/userinfo',
497         { headers: { authorization: 'Bearer ' + t2.body.access_token } });
498     b = await r.json();
499     r.status === 403 ? pass('403 ' + b.error + ' at /userinfo')
500         : fail('expected 403, got ' + r.status);
501     note('OAuth answers "what may this app do". OpenID Connect answers "who is");
502     note('this, and when did they prove it". You opt into the second with a scope.');
```

503 } };

504

505 // ----- Lab 12

506 LABS['agent-device'] = { title: 'An agent with no browser (device grant)',

507 run: async () => {

508 step(1, 'The agent asks for a code a human can type somewhere else');

509 why('It has no browser, so there is no redirect it could ever receive.');

510 const limits = [{ type: 'payment_initiation', actions: ['initiate'],

511 maxAmount: '100.00', fromAccount: 'ACC-4410' }];

512 const start = await post('/device_authorization', {

513 scope: 'openid accounts:read payments:write',

514 authorization_details: JSON.stringify(limits), AGENT);

515 if (start.status !== 200) return fail(JSON.stringify(start.body));

516 val('user_code', start.body.user_code);

517 val('verification_uri', start.body.verification_uri);

518 val('poll_interval', start.body.interval + 's');

519 why('The agent shows that code to the operator and starts polling.');

520

521 step(2, 'The agent polls before anyone has approved');

522 let t = await post('/token', { grant_type: DEVICE_GRANT,

523 device_code: start.body.device_code }, AGENT);

524 t.body.error === 'authorization_pending'

525 ? pass('authorization_pending - correct, it must keep waiting')

526 : fail(JSON.stringify(t.body));

527

528 step(3, 'Polling too fast is told to back off');

529 t = await post('/token', { grant_type: DEVICE_GRANT,

530 device_code: start.body.device_code }, AGENT);

531 t.body.error === 'slow_down' ? pass('slow_down - the interval is enforced')

532 : note('got ' + t.body.error);

533

534 step(4, 'The human approves on their own device');

535 const approve = await fetch(AS + '/device', form({

536 user_code: start.body.user_code, username: 'alice',

537 password: 'wonderland', action: 'allow' }));

538 (await approve.text()).includes('Approved')

539 ? pass('approved by alice, at the IdP, on a device with a screen')

540 : fail('approval failed');

541

542 step(5, 'The next poll returns tokens');

543 await pause(5200); // respect the advertised interval

The client, and all fifteen labs

LINES 544–597 OF 705

```

544   t = await post('/token', { grant_type: DEVICE_GRANT,
545     device_code: start.body.device_code }, AGENT);
546   if (t.status !== 200) return fail(JSON.stringify(t.body));
547   pass('tokens issued to the agent');
548   val('scope', t.body.scope);
549   val('refresh_token', t.body.refresh_token ? 'yes - it can work unattended' : 'none');
550   const claims = C.decodeJwt(t.body.access_token).payload;
551   val('sub', claims.sub + ' (the USER, not the agent)');
552   val('client_id', claims.client_id + ' (the agent)');
553   console.log('  authorization_details:',
554     JSON.stringify(claims.authorization_details));
555   note('The token acts for alice but records which agent holds it. That pair');
556   note('is what makes agent activity auditable.');
```

```

557   return t.body;
558 } };
559
560 // ----- Lab 13
561 LABS['agent-rar'] = { title: 'Structured limits a scope cannot express',
562   run: async () => {
563     const tok = await LABS['agent-device'].run();
564     if (!tok || !tok.access_token) return fail('could not get an agent token');
565     const at = tok.access_token;
566
567     step(6, 'A payment inside the granted limit');
568     let r = await fetch(API + '/api/payments', { method: 'POST',
569       headers: { authorization: 'Bearer ' + at,
570         'content-type': 'application/x-www-form-urlencoded' },
571       body: 'amount=50&from=ACC-4410' });
572     let b = await r.json();
573     r.status === 201 ? pass('201 - payment of 50 accepted')
574       : fail(r.status + ' ' + JSON.stringify(b));
575
576     step(7, 'The same token, a much larger payment');
577     r = await fetch(API + '/api/payments', { method: 'POST',
578       headers: { authorization: 'Bearer ' + at,
579         'content-type': 'application/x-www-form-urlencoded' },
580       body: 'amount=5000&from=ACC-4410' });
581     b = await r.json();
582     r.status === 403 ? pass('403 ' + b.error) : fail('expected 403, got ' + r.status);
583     why(b.error_description || '');
584
585     step(8, 'And from an account it was never granted');
586     r = await fetch(API + '/api/payments', { method: 'POST',
587       headers: { authorization: 'Bearer ' + at,
588         'content-type': 'application/x-www-form-urlencoded' },
589       body: 'amount=10&from=ACC-9982' });
590     b = await r.json();
591     r.status === 403 ? pass('403 ' + b.error) : fail('expected 403, got ' + r.status);
592     note('scope=payments:write was satisfied every time. The structured grant is');
593     note('what stopped it. This is why agents need RFC 9396, not bigger scopes.');
```

```

594   } };
595
596 // ----- Lab 14
597 LABS['agent-exchange'] = { title: 'Delegation chains and the act claim',

```

The client, and all fifteen labs

LINES 598–651 OF 705

```

598   run: async () => {
599     step(1, 'An agent obtains a user token (device grant, as in Lab 11));
600     const tok = await (async () => {
601       const s2 = await post('/device_authorization',
602         { scope: 'openid accounts:read' }, AGENT);
603       await fetch(AS + '/device', form({ user_code: s2.body.user_code,
604         username: 'alice', password: 'wonderland', action: 'allow' }));
605       await pause(5200);
606       return (await post('/token',
607         { grant_type: DEVICE_GRANT, device_code: s2.body.device_code }, AGENT)).body;
608     })();
609     if (!tok.access_token) return fail('no agent token');
610     val('agent token sub', C.decodeJwt(tok.access_token).payload.sub);
611
612     step(2, 'The agent hands it to a backend service, which must call the API');
613     why('That service must not pretend to BE alice, and must not act as itself');
614     why('either. It exchanges the token for one that records both.');
615     const ex = await post('/token', { grant_type: EXCHANGE_GRANT,
616       subject_token: tok.access_token,
617       subject_token_type: 'urn:ietf:params:oauth:token-type:access_token',
618       scope: 'accounts:read' }, ORCH);
619     if (ex.status !== 200) return fail(JSON.stringify(ex.body));
620     pass('exchanged');
621     const c = C.decodeJwt(ex.body.access_token).payload;
622     val('sub', c.sub + ' (still alice - the authority is hers)');
623     console.log(' act:', JSON.stringify(c.act) + ' <- who is acting');
624
625     step(3, 'The API sees the whole chain');
626     const r = await fetch(API + '/api/accounts',
627       { headers: { authorization: 'Bearer ' + ex.body.access_token } });
628     const b = await r.json();
629     val('status', r.status);
630     val('acting_chain', JSON.stringify(b.acting_chain));
631     note('Without `act`, that request would look exactly like alice at a browser.');
```

632 note('With it, an auditor can answer "which agent did this?" months later.');

```

633   } };
634
635   // ----- Lab 15
636   LABS['agent-deputy'] = { title: 'A confused deputy tries to escalate',
637     run: async () => {
638       step(1, 'An agent is delegated read access only');
639       const s2 = await post('/device_authorization',
640         { scope: 'openid accounts:read' }, AGENT);
641       await fetch(AS + '/device', form({ user_code: s2.body.user_code,
642         username: 'alice', password: 'wonderland', action: 'allow' }));
643       await pause(5200);
644       const tok = (await post('/token',
645         { grant_type: DEVICE_GRANT, device_code: s2.body.device_code }, AGENT)).body;
646       val('granted scope', tok.scope);
647
648       step(2, 'The backend tries to exchange it for permission to move money');
649       why('This is the confused deputy: a trusted service being talked into using');
650       why('authority it holds on behalf of someone who never granted it.');
```

651 const ex = await post('/token', { grant_type: EXCHANGE_GRANT,

The client, and all fifteen labs

LINES 652–705 OF 705

```

652     subject_token: tok.access_token,
653     subject_token_type: 'urn:ietf:params:oauth:token-type:access_token',
654     scope: 'accounts:read payments:write' }, ORCH);
655   ex.status !== 200 ? pass('REFUSED: ' + ex.body.error)
656                     : fail('escalation succeeded - delegation must only narrow');
657   why(ex.body.error_description || '');
658
659   step(3, 'Even the agent\'s own token cannot reach the payments endpoint');
660   const r = await fetch(API + '/api/payments', { method: 'POST',
661     headers: { authorization: 'Bearer ' + tok.access_token,
662       'content-type': 'application/x-www-form-urlencoded' },
663     body: 'amount=10' });
664   r.status === 403 ? pass('403 insufficient_scope at the API too')
665                   : fail('expected 403, got ' + r.status);
666   note('Two independent checks refused it: the AS would not widen the grant,');
667   note('and the API would not honour a scope the token never carried.');
```

```

668 } };
669
670 // =====
671 const ORDER = ['happy', 'pkce', 'attack', 'nopkce', 'replay', 'refresh',
672   'scope', 'idtoken', 'm2m', 'revoke',
673   'oidc', 'agent-device', 'agent-rar', 'agent-exchange',
674   'agent-deputy'];
675
676 async function main() {
677   const which = process.argv[2] || 'list';
678
679   if (which === 'list') {
680     console.log('\n' + B + ' OAuth 2.1 + PKCE labs' + R + '\n');
681     ORDER.forEach((k, i) => console.log(' ' + String(i + 1).padStart(2)
682       + ' ' + k.padEnd(15) + LABS[k].title));
683     console.log('\n node client.js <name> | node client.js all\n');
684     return;
685   }
686
687   // Both servers must be up before any lab can run.
688   for (const [name, url] of [['authorization server', AS], ['api', API]]) {
689     try { await fetch(url + '/healthz'); }
690     catch { console.log(RE + '\n Cannot reach the ' + name + ' at ' + url
691       + '\n Start it first: node ' + (url === AS ? 'authz-server'
692         : 'api-server') + '.js\n' + R); process.exit(1); }
693   }
694
695   const names = which === 'all' ? ORDER : [which];
696   for (const n of names) {
697     if (!LABS[n]) { console.log('Unknown lab: ' + n); process.exit(2); }
698     head('LAB ' + (ORDER.indexOf(n) + 1) + ' ' + LABS[n].title);
699     await LABS[n].run();
700   }
701   console.log('\n' + D + ' Watch the server side of all of this in the terminal'
702     + ' running authz-server.js\n' + R);
703 }
704
705 main().catch((e) => { console.error(e); process.exit(1); });

```

Glossary

Every term used in this guide, in one place.

access token

A credential saying what an application may do, on whose behalf, at which API, until when. Usually a signed JWT. Sent to the resource server.

authorization code

A short-lived, single-use reference handed to the client through the browser, exchanged at the token endpoint for tokens. Worthless on its own.

authorization server (AS)

The service that authenticates the user, collects consent, and issues tokens. Publishes its metadata and public keys.

aud — the “audience” claim: which recipient a token is for. A resource server must reject tokens naming anyone else.

back channel

A direct server-to-server HTTPS call, invisible to the browser. Safe for secrets. See Chapter 3.

bearer token

A token usable by whoever holds it, with no further proof required — like a password. Hence short lifetimes.

client

The application requesting access. **Confidential** if it can keep a secret (server-side), **public** if it cannot (SPA, mobile, CLI).

code challenge / code verifier

The PKCE pair. The verifier is a random secret held by the client; the challenge is its SHA-256 hash, safe to send through the browser.

front channel

Communication via browser redirects. Visible in URL bars, history, logs and `Referer` headers. Never carries secrets.

grant type

How a client obtains a token. OAuth 2.1 keeps `authorization_code`, `refresh_token` and `client_credentials`; `implicit` and `password` are gone.

identity provider (IdP)

The service holding user accounts and performing authentication. Usually packaged with the authorization server, but a distinct job.

ID token

An OpenID Connect JWT stating who signed in and when, addressed to the client. Not an API credential.

introspection — RFC 7662: an endpoint where a resource server can ask whether a token is still active. Sees revocation immediately.

iss — the “issuer” claim: which authorization server minted a token. Checked on every validation.

JWKS — JSON Web Key Set: the public keys an authorization server publishes so anyone can verify its signatures offline.

JWT

JSON Web Token: header, payload and signature, base64url-encoded and dot-separated. Signed, **not** encrypted.

kid — key id: names which published key signed a token, so keys can be rotated without downtime.

nonce

A random value echoed inside the ID token, proving it was minted for this login.

PKCE

Proof Key for Code Exchange (RFC 7636). Binds an authorization code to the application that requested it. Mandatory in OAuth 2.1.

refresh token

Exchanged for new access tokens without user interaction, and rotated on every use.

resource server

The API holding the data. Validates tokens and enforces scope.

scope

A label for a slice of authority (`accounts:read`): requested, capped by registration and entitlement, then approved by the user.

state

A random per-request value echoed back on the callback, so a client can recognise a flow it started.

The specifications

Which document says what, and a sane order to read them in.

Read these, in this order

DOCUMENT	WHAT IT GIVES YOU
RFC 9700 OAuth 2.0 Security BCP	Start here. The accumulated lessons of a decade of deployments, with the attacks explained. Almost everything OAuth 2.1 requires is justified in this document.
OAuth 2.1 draft-ietf-oauth-v2-1	The consolidated core: one specification with the dangerous options already removed. Read it after 9700 and it will feel inevitable.
RFC 7636 PKCE	Short and readable. The definition of <code>code_verifier</code> , <code>code_challenge</code> and <code>S256</code> , and the attack that motivated them.
RFC 6749 OAuth 2.0 core	The original framework. Still the normative text for most parameter definitions — but read it knowing which parts 2.1 deletes.
RFC 6750 Bearer token usage	How to send a token and what each error code means. This is where <code>401</code> vs <code>403 insufficient_scope</code> is defined.

The ones you will need eventually

DOCUMENT	WHAT IT COVERS
RFC 8414	Authorization server metadata: <code>/.well-known/oauth-authorization-server</code> , so clients discover endpoints instead of hard-coding them.
RFC 9068	JWT access tokens: the <code>at+jwt</code> type and the claims an API should expect.
RFC 7662 / 7009	Token introspection and revocation.
RFC 8252	OAuth for native apps: system browsers, custom URI schemes, and why an embedded web view is the wrong answer.
RFC 8707 / 9207	Resource indicators (audience selection) and the <code>iss</code> authorization-response parameter.
RFC 9449	DPoP: sender-constrained tokens, so a stolen token is useless without the client's key. The direction bearer tokens are heading.
OpenID Connect Core	Authentication on top of OAuth: ID tokens, <code>nonce</code> , <code>/userinfo</code> and the <code>openid</code> scope.

One last piece of advice. Do not write your own OAuth client or server for production — use a maintained library and a real authorization server. The value of understanding the protocol is in configuring it correctly, reading the error you actually got, and noticing when somebody proposes quietly removing one of these rules.

This guide accompanies the OAuth 2.1 + PKCE Simulator in `oauth-simulator/`. Regenerate it with `make guide` from `dev-guide/src/dev-guide.html`.